

Интеграция обучения с подкреплением и планирования

Алексей Скрынник

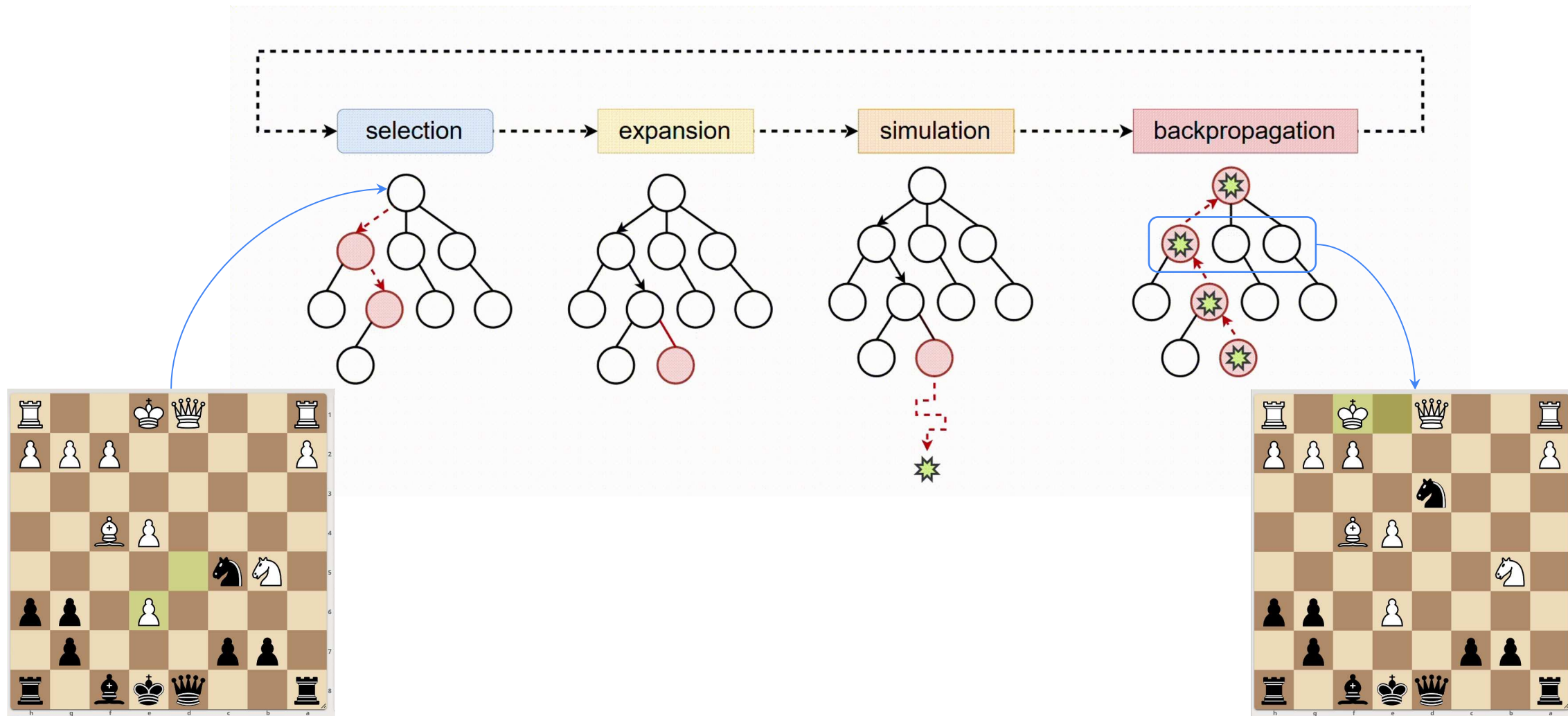
План

- ▶ Монте Карло поиск по дереву
- ▶ AlphaZero
- ▶ MuZero
- ▶ Применения и улучшения
- ▶ DREAMER

01

Монте Карло поиск по дереву

Background: Monte-Carlo Tree Search (MCTS)



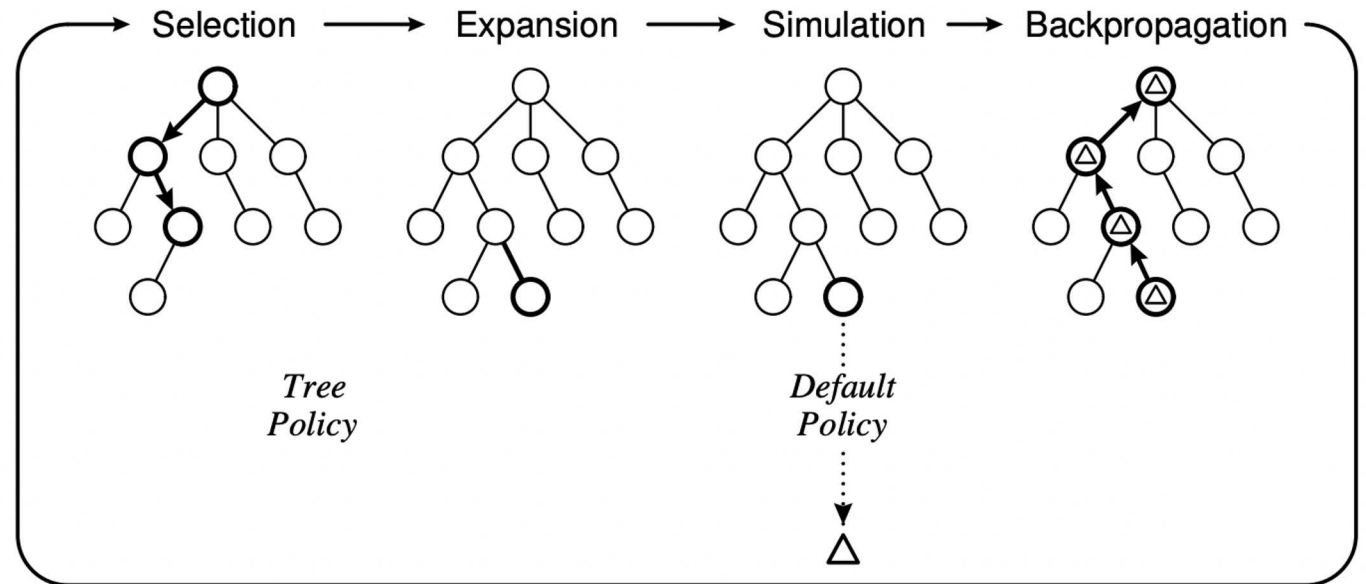
Background: Monte-Carlo Tree Search (MCTS)



How to make single action?

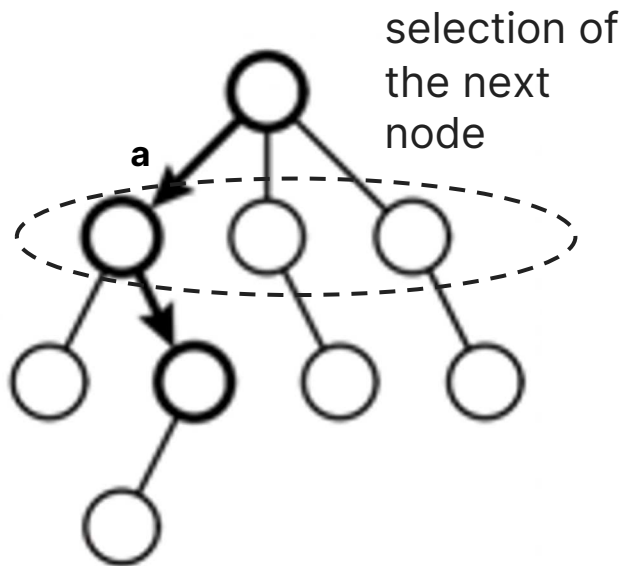
Algorithm 1 General MCTS approach.

```
function MCTSSEARCH( $s_0$ )  
  create root node  $v_0$  with state  $s_0$   
  while within computational budget do  
     $v_l \leftarrow \text{TREEPOLICY}(v_0)$   
     $\Delta \leftarrow \text{DEFAULTPOLICY}(s(v_l))$   
    BACKUP( $v_l, \Delta$ )  
  return  $a(\text{BESTCHILD}(v_0))$ 
```



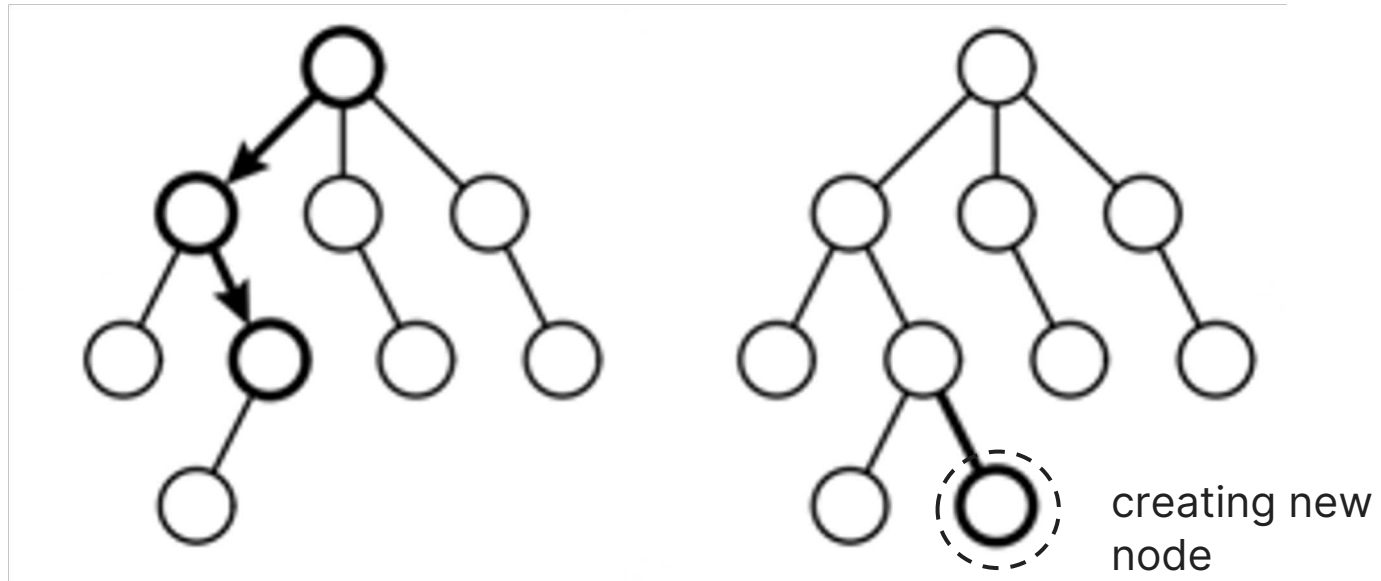
A Survey of Monte Carlo Tree Search Methods

MCTS: Selection

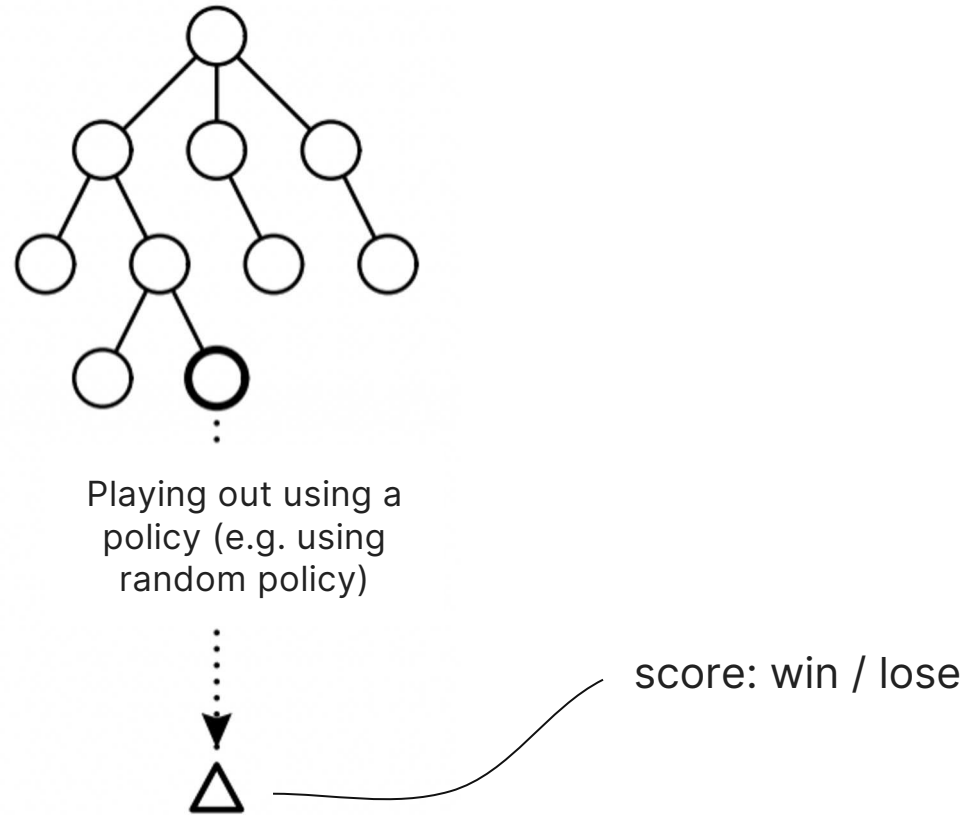


$$UCT(s, a) = \underbrace{Q(s, a)}_{\frac{w_a}{N(s, a)} \text{ \% wins}} + c \cdot \sqrt{\frac{\overbrace{\ln(\sum_b N(s, b))}^{\text{number of visits of parent node}}}{\underbrace{1 + N(s, a)}_{\text{number of visits for node}}}}$$

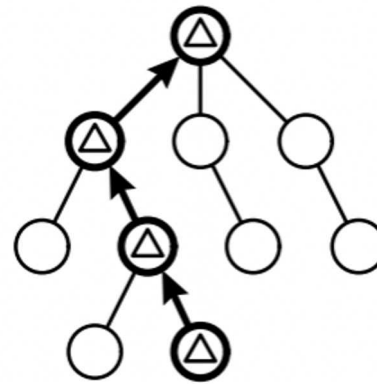
MCTS: Expansion



MCTS: Simulation



MCTS: Backpropagation



update

$$N(s, a) \leftarrow N(s, a) + 1$$

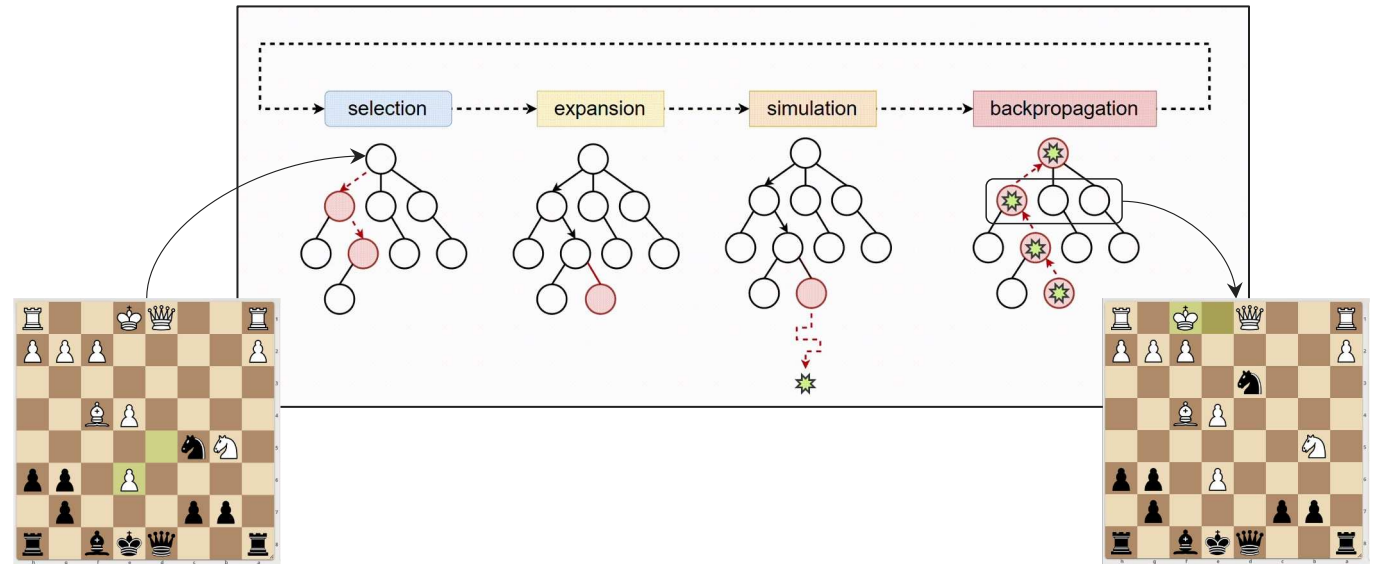
$$w_a \leftarrow w_a + \text{score}$$

Monte-Carlo Tree Search

We call Algorithm 1 for each individual action in the episode

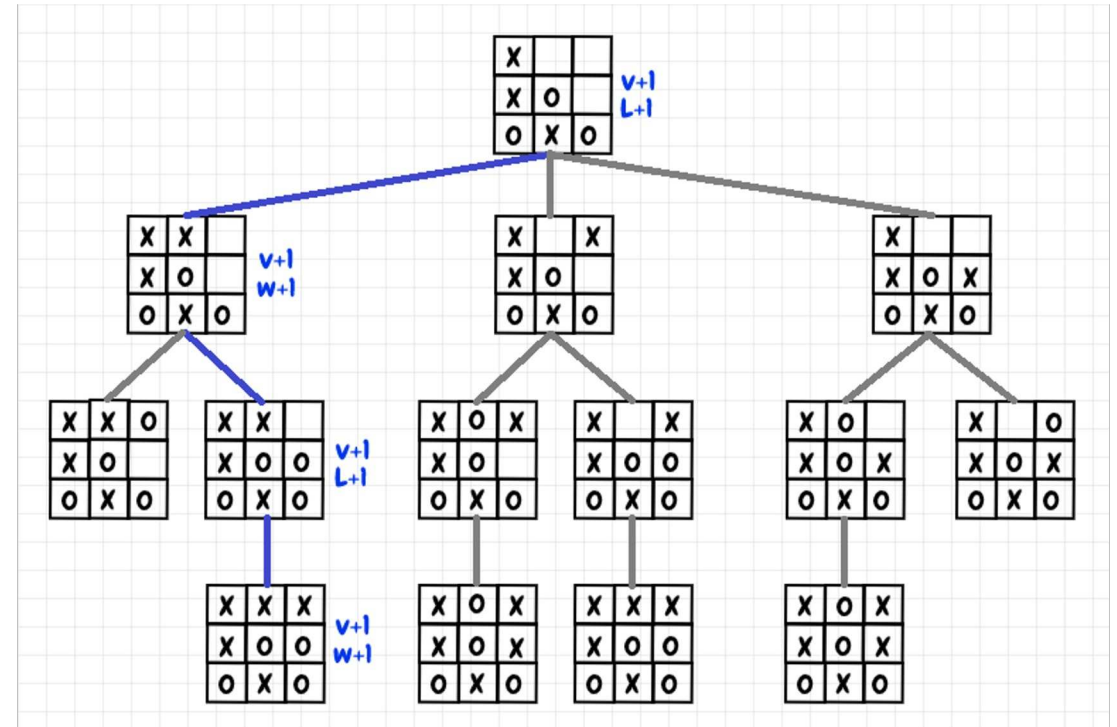
Algorithm 1 General MCTS approach.

```
function MCTSSEARCH( $s_0$ )  
  create root node  $v_0$  with state  $s_0$   
  while within computational budget do  
     $v_l \leftarrow \text{TREEPOLICY}(v_0)$   
     $\Delta \leftarrow \text{DEFAULTPOLICY}(s(v_l))$   
     $\text{BACKUP}(v_l, \Delta)$   
  return  $a(\text{BESTCHILD}(v_0))$ 
```



Classic MCTS Summary:

- Smart alternative to exhaustive search
- MCTS has been used for various tasks, such as planning, scheduling, and physical simulations (e.g., the HOOT algorithm)
- It gained the most recognition for its application to the game of Go
- It can show strong performance with a high-quality default policy (for limited action space)



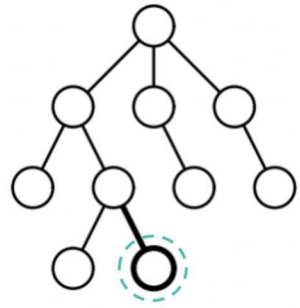
02

AlphaZero

AlphaZero – этапы построения дерева

1

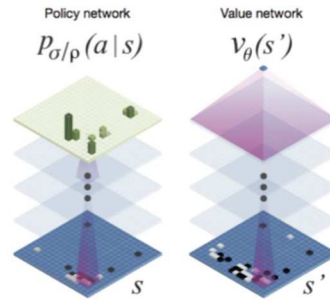
How to choose actions in the tree?



$Q + U$

The mean value
of the next state

A function of **P** and **N**
that increases if an action
hasn't been explored much,
relative to the other
actions,
or if the prior probability
of the action is high



value from critic

$$pUCT = Q(s, a) + P(s, a) \cdot \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \left(c_1 + \log \left(\frac{\sum_b N(s, b) + c_2 + 1}{c_2} \right) \right)$$

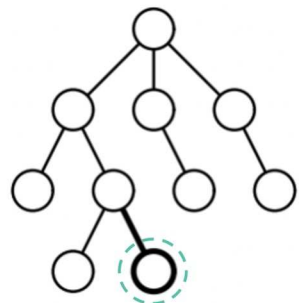
Prior probability
of an action, determined
by the actor's policy

exploration

AlphaZero – этапы построения дерева

1

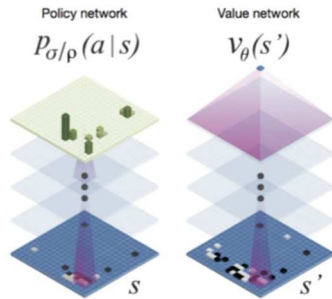
How to choose actions in the tree?



$Q + U$

The mean value
of the next state

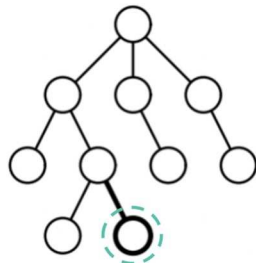
A function of $\mathbf{P}$ and $\mathbf{N}$
that increases if an action
hasn't been explored much,
relative to the other
actions,
or if the prior probability
of the action is high



$$pUCT = \underbrace{Q(s, a)}_{\text{value from critic}} + \underbrace{P(s, a)}_{\text{Prior probability of an action, determined by the actor's policy}} \cdot \underbrace{\frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \left(c_1 + \log \left(\frac{\sum_b N(s, b) + c_2 + 1}{c_2} \right) \right)}_{\text{exploration}}$$

2

We continue until we find an unexplored node



The game state of the leaf node is passed into the neural network, which outputs predictions about two things:

$\mathbf{P}$ — move probabilities

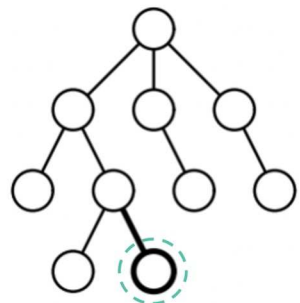
$\mathbf{V}$ — value of the state (for the current player)

The move probabilities p are attached to the new feasible actions from the leaf node

AlphaZero – этапы построения дерева

1

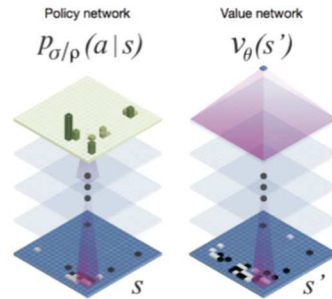
How to choose actions in the tree?



$Q + U$

The mean value of the next state

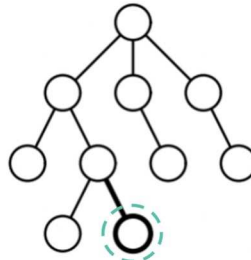
A function of P and N that increases if an action hasn't been explored much, relative to the other actions, or if the prior probability of the action is high



$$pUCT = \underbrace{Q(s, a)}_{\text{value from critic}} + \underbrace{P(s, a)}_{\text{Prior probability of an action, determined by the actor's policy}} \cdot \underbrace{\frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \left(c_1 + \log \left(\frac{\sum_b N(s, b) + c_2 + 1}{c_2} \right) \right)}_{\text{exploration}}$$

2

We continue until we find an unexplored node



The game state of the leaf node is passed into the neural network, which outputs predictions about two things:

P — move probabilities

V — value of the state (for the current player)

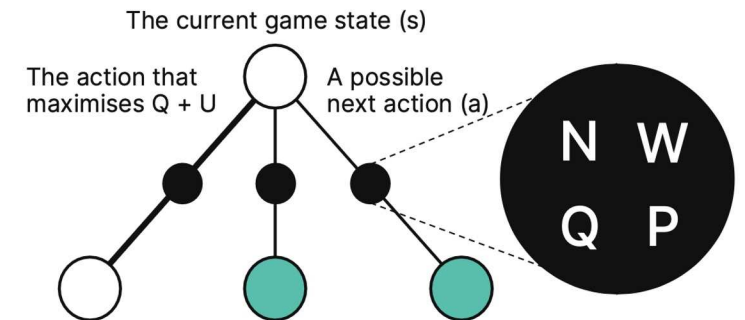
The move probabilities p are attached to the new feasible actions from the leaf node

3

Backpropagation

Each edge that was traversed to get to the leaf node is updated as follows:

$$\begin{aligned} N &\rightarrow N + 1 \\ W &\rightarrow W + 1 \\ Q &= W / N \end{aligned}$$



How to choose the best action after constructing the tree?

...then select a move

After 1,600 simulations, the move can either be chosen:

Deterministically (for competitive play)

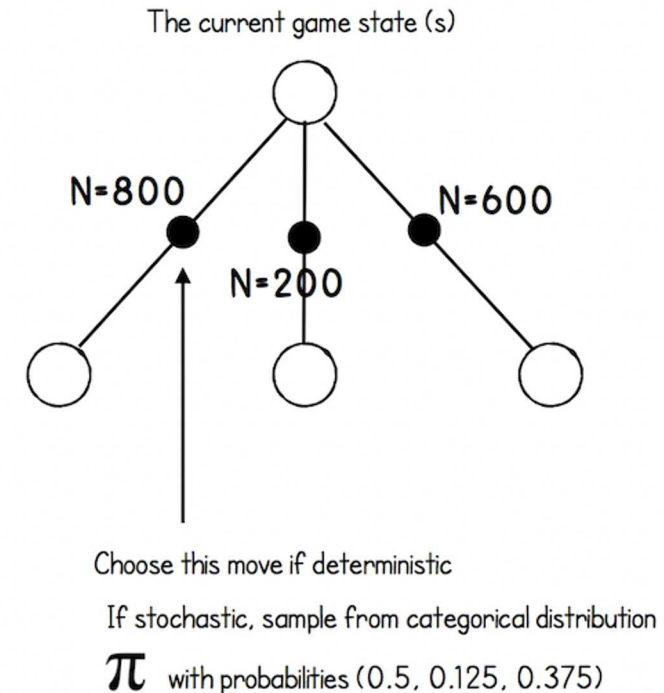
Choose the action from the current state **a) the most explored subtree**

Stochastically (for exploratory play)

Choose the action from the current state from the distribution

$$\pi \sim N^{1/\tau}$$

where τ is a temperature **b) sampling from distribution**



Tree search is a policy improvement operator!
After N simulations, we get a better policy.

How is it trained?

It's implicit curriculum learning!

SELF PLAY

Create a 'training set'

Playing against
its own copy

At each move, the following information is stored



State

π

The search probabilities
(from the MCTS)

Game results



The winner
(+1 if this player won, -1 if
this player lost - added once
the game has finished)

Improved by MCTS policy
(softmax of probabilities)

How is it trained?

It's implicit curriculum learning!

SELF PLAY

Create a 'training set'

Playing against
its own copy

At each move, the following information is stored



State

π

The search probabilities
(from the MCTS)

Game results



The winner
(+1 if this player won, -1 if
this player lost - added once
the game has finished)

RETRAIN NETWORK

Optimise the network weights **learning**

A TRAINING LOOP

Sample a mini-batch of 2048 positions from the last 500,000 games

Retrain the current neural network on these positions

– The game states are the input (see 'Deep Neural Network Architecture')

Loss function

Compares predictions from the neural network with the search probabilities and actual winner

PREDICTIONS

$\mathbf{p}$
 $\mathbf{v}$

Cross-entropy

+

Mean-squared error

+

Regularisation

π



ACTUAL

After every 1,000 training loops, evaluate the network

$$l = (z - v)^2 - \pi^T \log \mathbf{p} + c ||\theta||^2$$

critic MSE actor cross-entropy L2 reg

Improved by MCTS policy
(softmax of probabilities)

How is it trained?

It's implicit curriculum learning!

SELF PLAY

Create a 'training set'

Playing against its own copy

At each move, the following information is stored



State

π

The search probabilities
(from the MCTS)

Game results



The winner
(+1 if this player won, -1 if this player lost - added once the game has finished)

RETRAIN NETWORK

Optimise the network weights **learning**

A TRAINING LOOP

Sample a mini-batch of 2048 positions from the last 500,000 games

Retrain the current neural network on these positions

- The game states are the input (see 'Deep Neural Network Architecture')

Loss function

Compares predictions from the neural network with the search probabilities and actual winner

PREDICTIONS

$\mathbf{p}$
 $\mathbf{v}$

Cross-entropy

+

Mean-squared error

+

Regularisation

π



ACTUAL

After every 1,000 training loops, evaluate the network

$$l = (z - v)^2 - \pi^T \log \mathbf{p} + c ||\theta||^2$$

critic MSE actor cross-entropy L2 reg

EVALUATE NETWORK

Test to see if the new network is stronger

Play 400 games between the latest neural network and the current best neural network

Both players use MCTS to select their moves, with their respective neural networks to evaluate leaf nodes

Latest player must win 55% of games to be declared the new best player



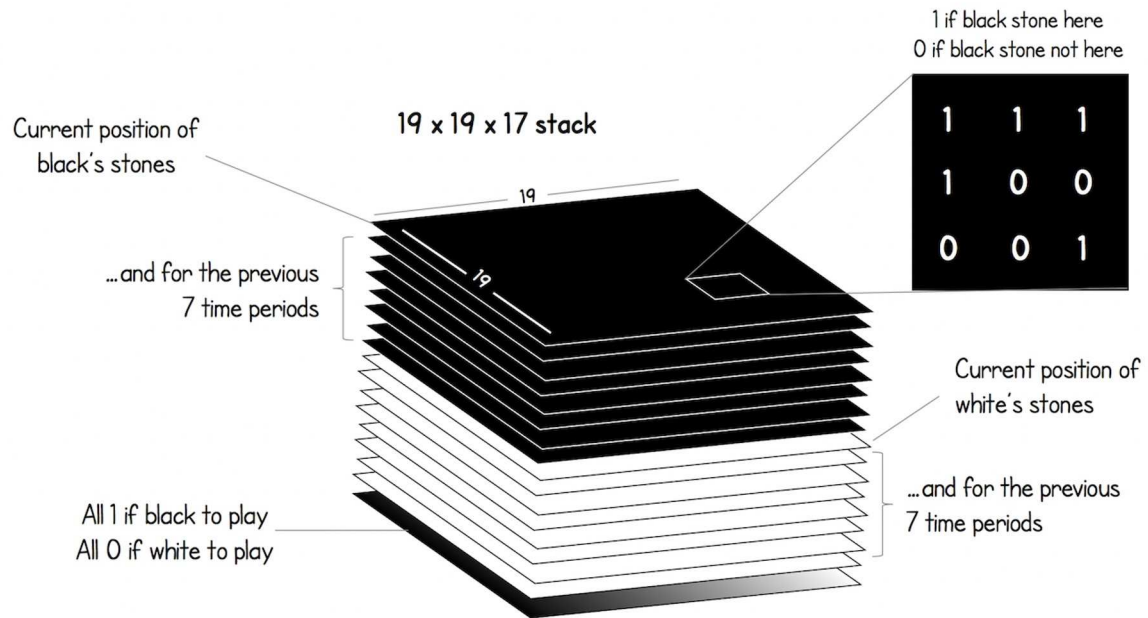
Policy estimation
(pitting against previous copy)

Improved by MCTS policy
(softmax of probabilities)

ResNet Encoder

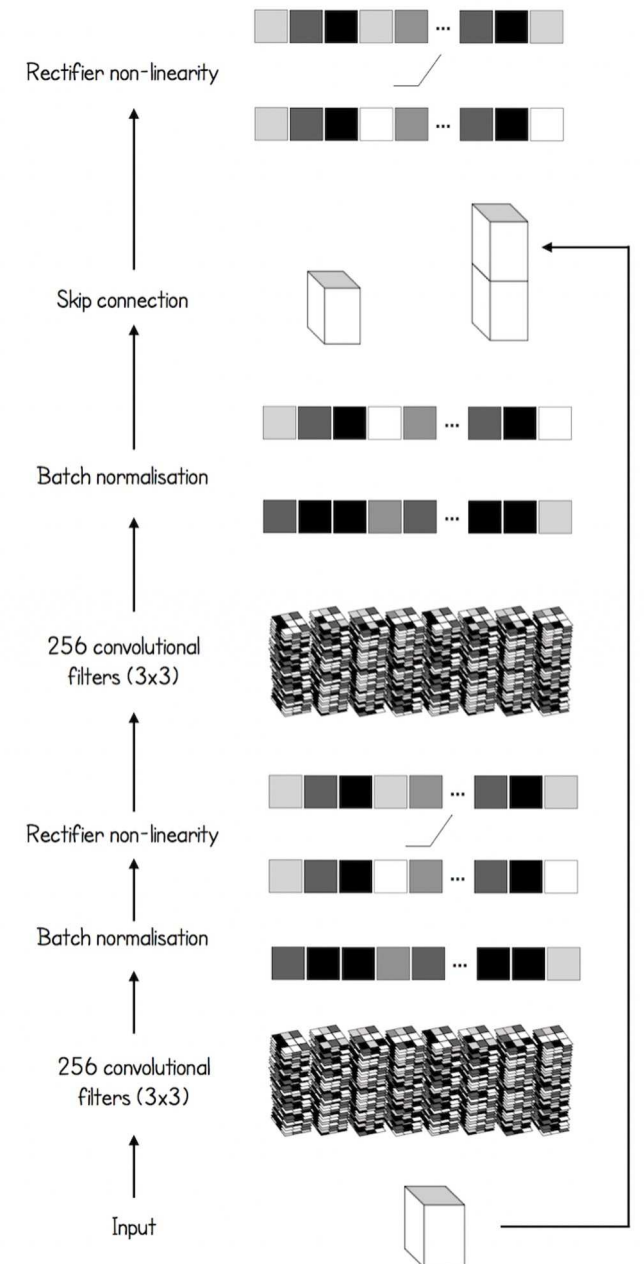
Game state encoding

WHAT IS A 'GAME STATE'



This stack is the input to the deep neural network

Default ResNet without
max-pooling



Results

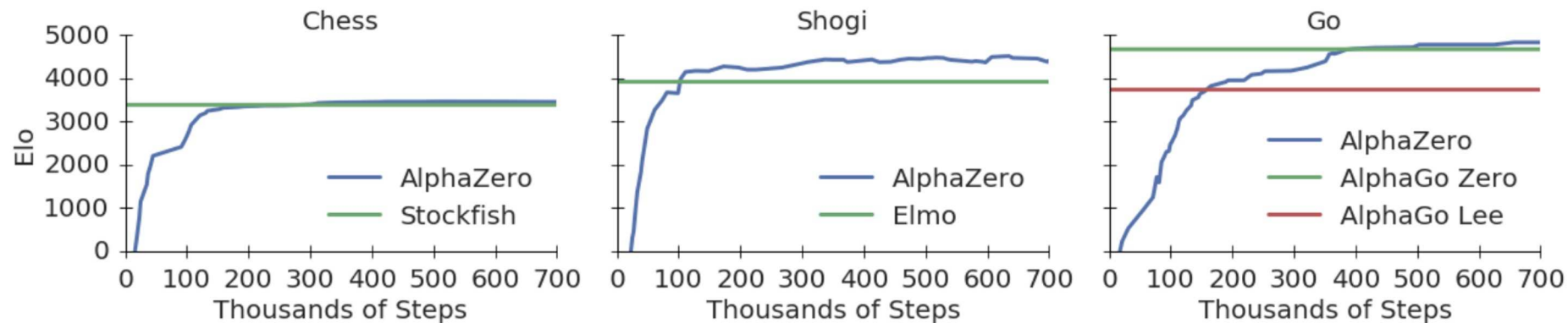
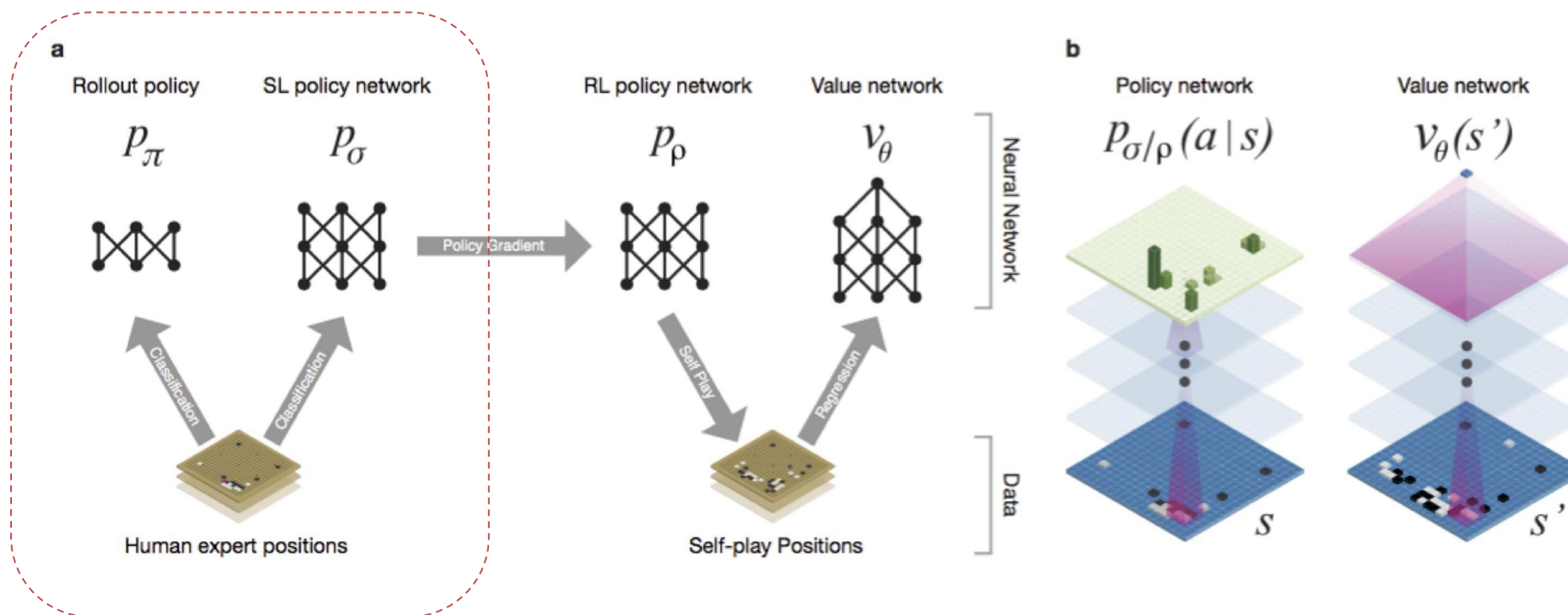


Figure 1: Training *AlphaZero* for 700,000 steps. Elo ratings were computed from evaluation games between different players when given one second per move. **a** Performance of *AlphaZero* in chess, compared to 2016 TCEC world-champion program *Stockfish*. **b** Performance of *AlphaZero* in shogi, compared to 2017 CSA world-champion program *Elmo*. **c** Performance of *AlphaZero* in Go, compared to *AlphaGo Lee* and *AlphaGo Zero* (20 block / 3 day) (29).

[Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm](#)

Difference with AlphaGo



Pretraining: Behavioral cloning on human experts data

+ many other tricks, such as the fast rollout policy, which were later dropped

AlphaZero Summary:

- Tabula rasa self-play training
- No human knowledge input
- ResNet encoder architecture
- Superhuman performance, mastery in chess, shogi, and Go
- Limitations?



AlphaZero Summary:

- Tabula rasa self-play training
- No human knowledge input
- ResNet encoder architecture
- Superhuman performance, mastery in chess, shogi, and Go
- Requires a simulator
- Only deterministic environments
- Simplified RL (no discount factor, etc.)
- Computationally expensive



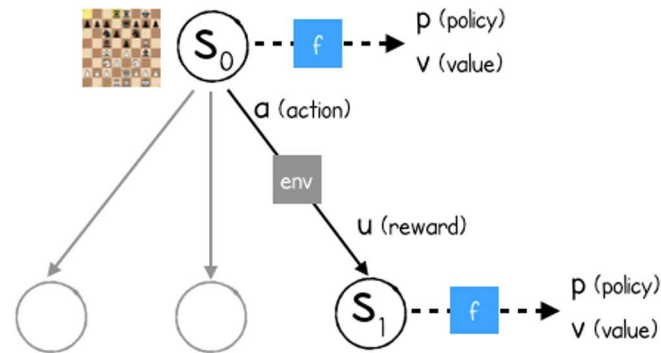
03



MuZero

AlphaZero vs MuZero

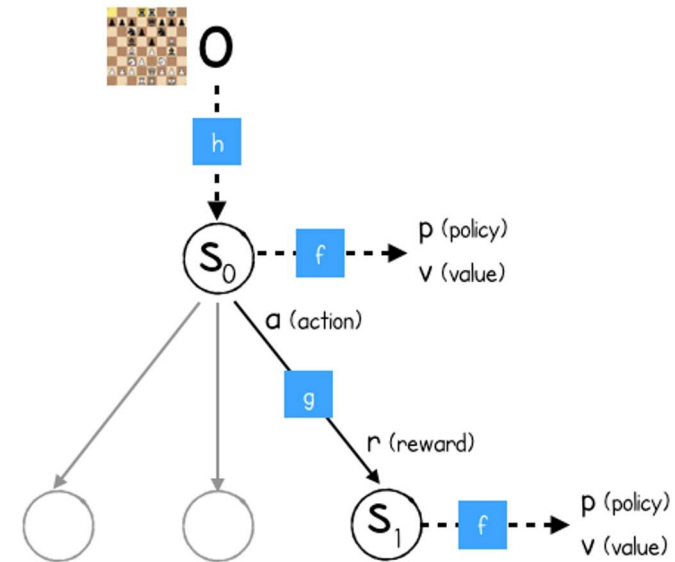
AlphaZero



AlphaZero has 1 network

prediction f : $\begin{matrix} \text{from} & \text{to} \\ s & \rightarrow p, v \end{matrix}$

MuZero



MuZero has 3 networks

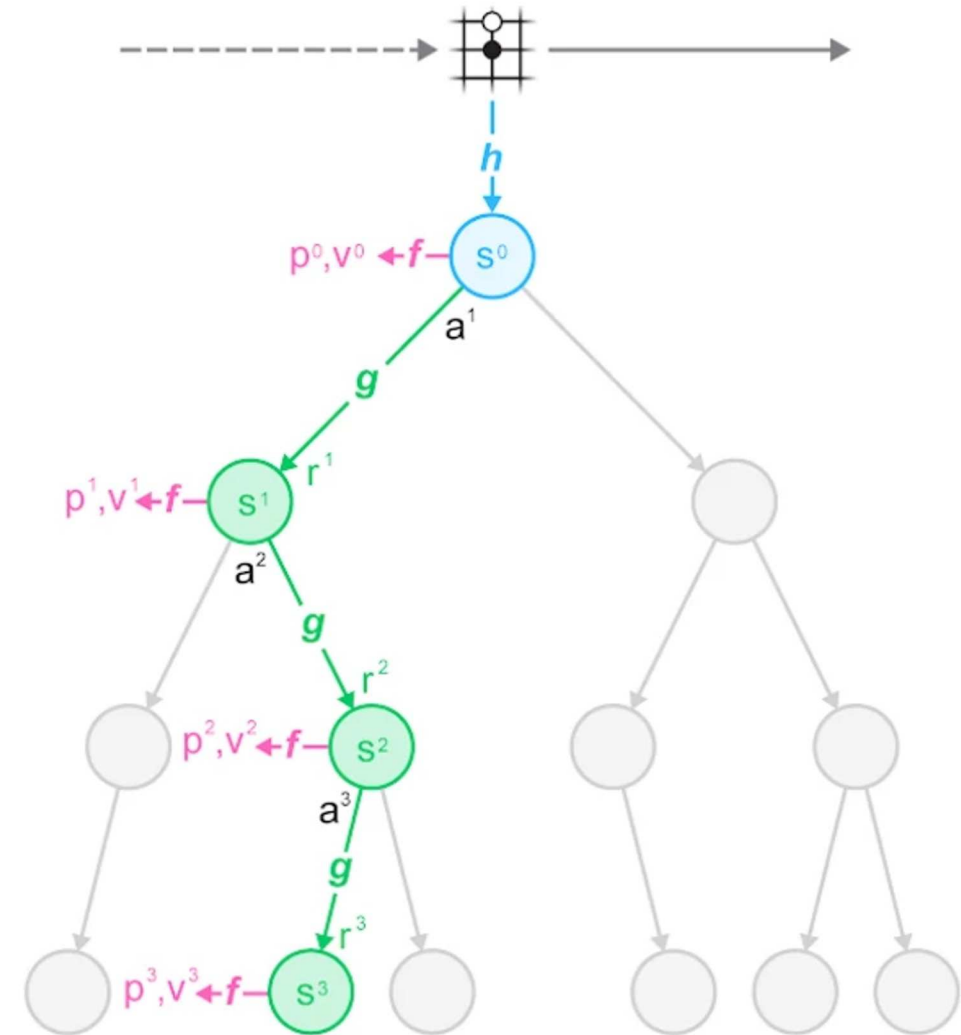
		from	to
prediction	f :	$s \rightarrow$	p, v
dynamics	g :	$s, a \rightarrow$	r, s
representation	h :	$o \rightarrow$	s

MuZero

representation $h_{\theta}(o_1, \dots, o_t) = s^0$

prediction $f_{\theta}(s^k) = \mathbf{p}^k, v^k$

dynamics $g_{\theta}(s^{k-1}, a^k) = r^k, s^k$



УСТ для RL

исследование

$$U(s, a) = r(s, a) + \gamma \cdot v(s') + c \cdot p(s, a)$$

дополнительная
нормализация

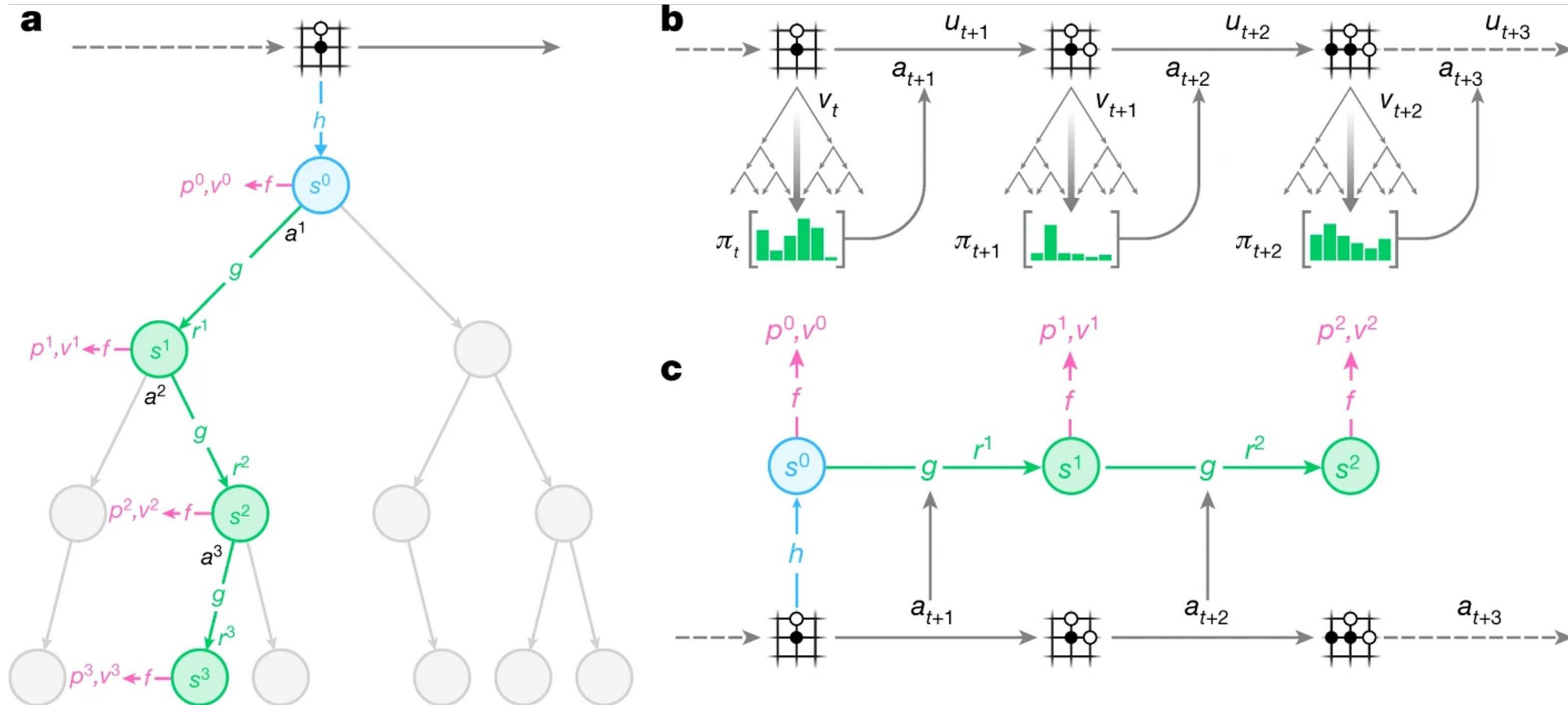
$$U(s, a) = \frac{r(s, a) + \gamma \cdot v(s') - q_{min}}{q_{max} - q_{min}} + c \cdot p(s, a)$$

MuZero

a – как планирует

b – как действует в среде

c – как обучает модель



MuZero Training

Model

$$\left. \begin{aligned} s^0 &= h_\theta(o_1, \dots, o_t) \\ r^k, s^k &= g_\theta(s^{k-1}, a^k) \\ \mathbf{p}^k, v^k &= f_\theta(s^k) \end{aligned} \right\} \mathbf{p}^k, v^k, r^k = \mu_\theta(o_1, \dots, o_t, a^1, \dots, a^k)$$

h produces s^0 , g is used to generate r^k and s^k ,
 s^0 and s^k are implicit targets

Search

$$\nu_t, \pi_t = MCTS(s_t^0, \mu_\theta)$$

$$a_t \sim \pi_t$$

Learning Rule

$$\mathbf{p}_t^k, v_t^k, r_t^k = \mu_\theta(o_1, \dots, o_t, a_{t+1}, \dots, a_{t+k})$$

$$z_t = \begin{cases} u_T & \text{for games} \\ u_{t+1} + \gamma u_{t+2} + \dots + \gamma^{n-1} u_{t+n} + \gamma^n \nu_{t+n} & \text{for general MDPs} \end{cases}$$

n-step bootstrapping

$$l_t(\theta) = \sum_{k=0}^K l^r(u_{t+k}, r_t^k) + l^v(z_{t+k}, v_t^k) + l^p(\pi_{t+k}, \mathbf{p}_t^k) + c \|\theta\|^2$$

reward critic actor L2 reg

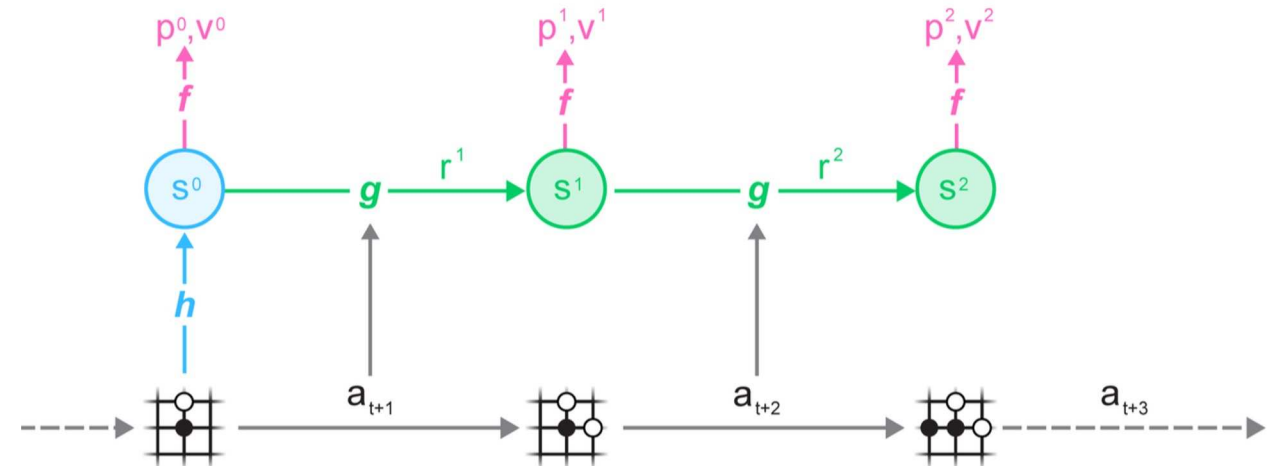
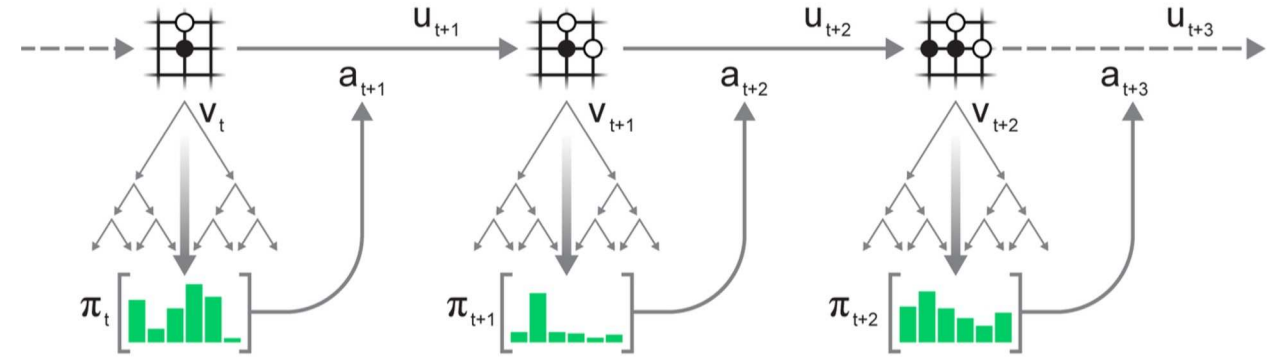
Losses

$$l^r(u, r) = \begin{cases} 0 & \text{for games} \\ \phi(u)^T \log \mathbf{r} & \text{for general MDPs} \end{cases}$$

$$l^v(z, q) = \begin{cases} (z - q)^2 & \text{for games} \\ \phi(z)^T \log \mathbf{q} & \text{for general MDPs} \end{cases}$$

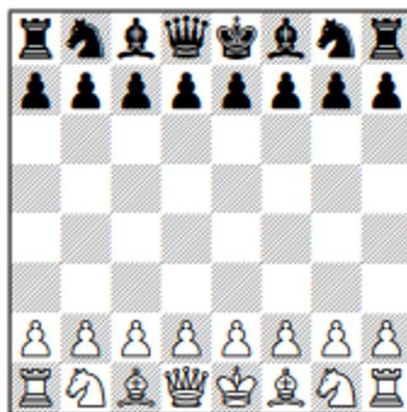
(phi is a cross-entropy over bins)

$$l^p(\pi, p) = \pi^T \log \mathbf{p}$$



Результаты

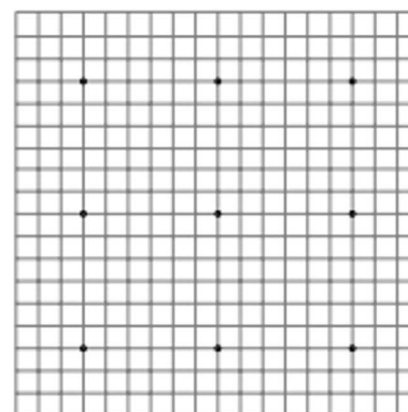
Chess



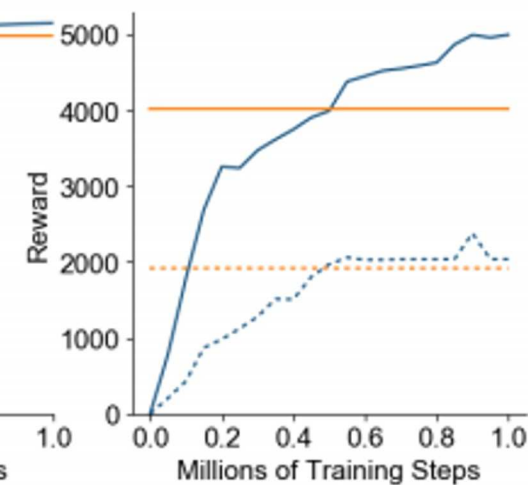
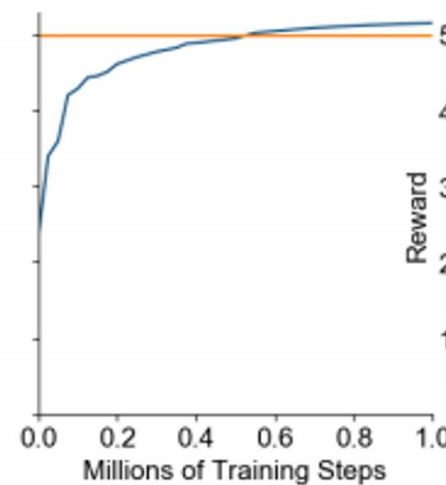
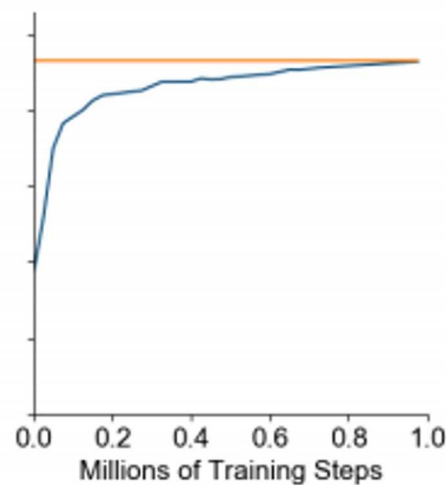
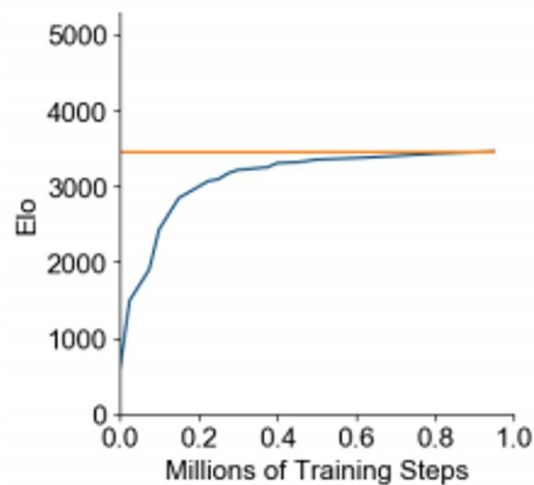
Shogi



Go



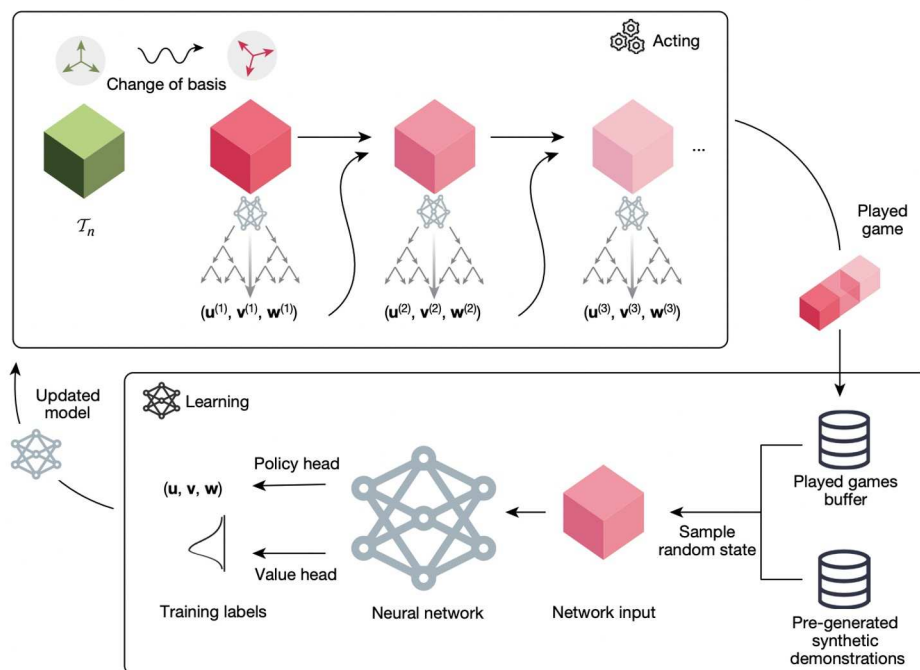
Atari



04

Применения и улучшения

Applications: Efficient Matrix Multiplication



$n \times m$ with $m \times p$ matrices

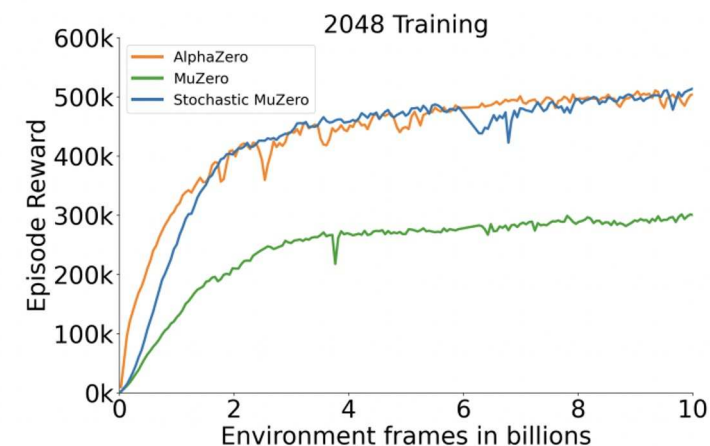
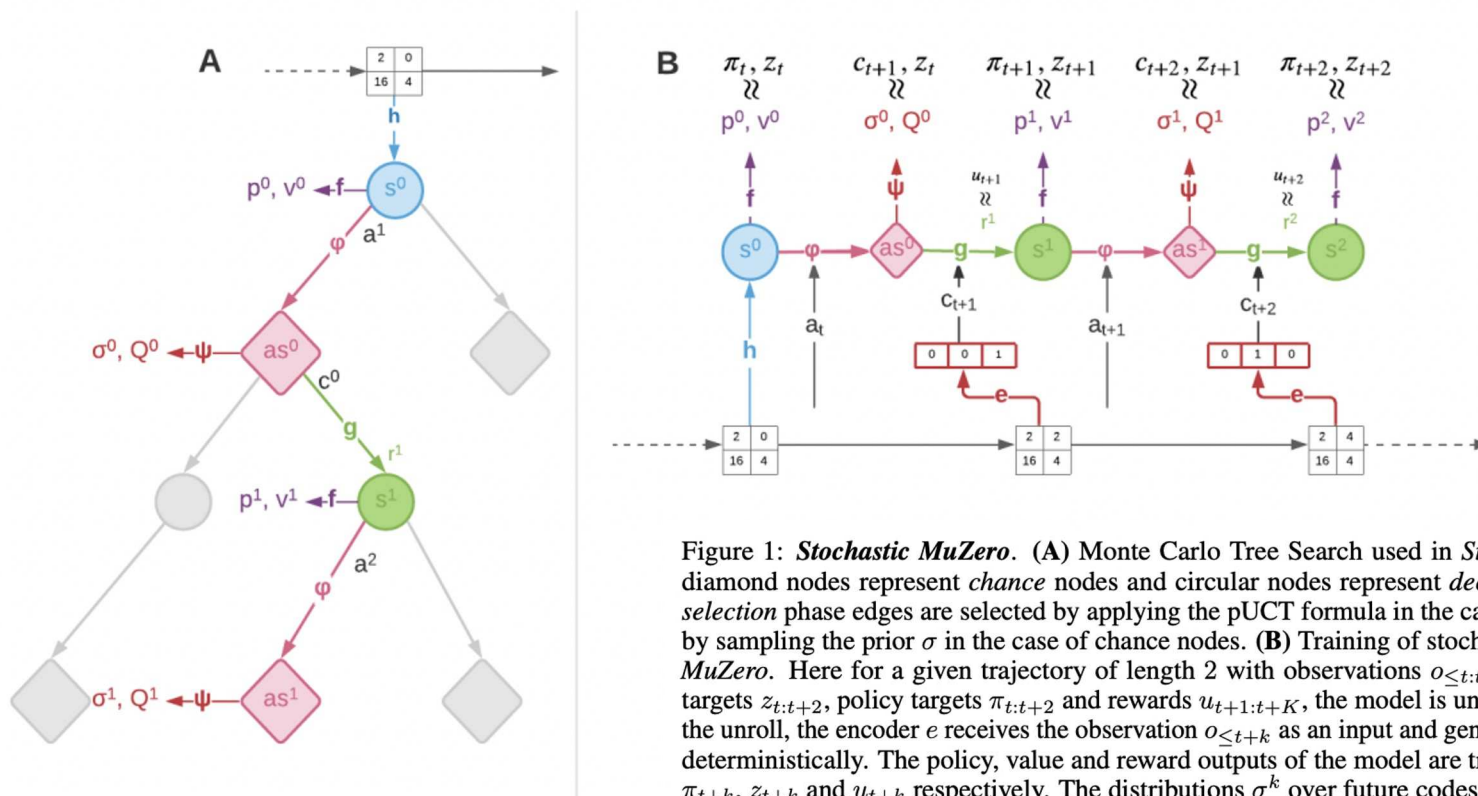
Size (n, m, p)	Best method known	Best rank known	AlphaTensor rank Modular Standard
(2, 2, 2)	(Strassen, 1969) ²	7	7
(3, 3, 3)	(Laderman, 1976) ¹⁵	23	23
(4, 4, 4)	(Strassen, 1969) ²	49	47
(5, 5, 5)	(3, 5, 5) + (2, 5, 5)	98	96
(2, 2, 3)	(2, 2, 2) + (2, 2, 1)	11	11
(2, 2, 4)	(2, 2, 2) + (2, 2, 2)	14	14
(2, 2, 5)	(2, 2, 2) + (2, 2, 3)	18	18
(2, 3, 3)	(Hopcroft and Kerr, 1971) ¹⁶	15	15
(2, 3, 4)	(Hopcroft and Kerr, 1971) ¹⁶	20	20
(2, 3, 5)	(Hopcroft and Kerr, 1971) ¹⁶	25	25
(2, 4, 4)	(Hopcroft and Kerr, 1971) ¹⁶	26	26
(2, 4, 5)	(Hopcroft and Kerr, 1971) ¹⁶	33	33
(2, 5, 5)	(Hopcroft and Kerr, 1971) ¹⁶	40	40
(3, 3, 4)	(Smirnov, 2013) ¹⁸	29	29
(3, 3, 5)	(Smirnov, 2013) ¹⁸	36	36
(3, 4, 4)	(Smirnov, 2013) ¹⁸	38	38
(3, 4, 5)	(Smirnov, 2013) ¹⁸	48	47
(3, 5, 5)	(Sedoglavic and Smirnov, 2021) ¹⁹	58	58
(4, 4, 5)	(4, 4, 2) + (4, 4, 3)	64	63
(4, 5, 5)	(2, 5, 5) $\otimes$ (2, 1, 1)	80	76

AlphaTensor – 2019

Discovering faster matrix multiplication algorithms with reinforcement learning

Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatin, Alexander Novikov, Francisco J. R. Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, David Silver, Demis Hassabis & Pushmeet Kohli

Applications: Stochastic Environments



2			4
		4	16
	8	16	32
8	16	32	64

Figure 1: **Stochastic MuZero.** (A) Monte Carlo Tree Search used in *Stochastic MuZero*, where diamond nodes represent *chance* nodes and circular nodes represent *decision* nodes. During the *selection* phase edges are selected by applying the pUCT formula in the case of decision nodes, and by sampling the prior σ in the case of chance nodes. (B) Training of stochastic model in *Stochastic MuZero*. Here for a given trajectory of length 2 with observations $o_{\leq t:t+2}$, actions $a_{t:t+2}$, value targets $z_{t:t+2}$, policy targets $\pi_{t:t+2}$ and rewards $u_{t+1:t+K}$, the model is unrolled for 2 steps. During the unroll, the encoder e receives the observation $o_{\leq t+k}$ as an input and generates a chance code c_{t+k} deterministically. The policy, value and reward outputs of the model are trained towards the targets π_{t+k} , z_{t+k} and u_{t+k} respectively. The distributions σ^k over future codes are trained to predict the code produced by the encoder.

Planning in Stochastic Environments with a Learned Model

Mujoco (DeepMind Control Suite)

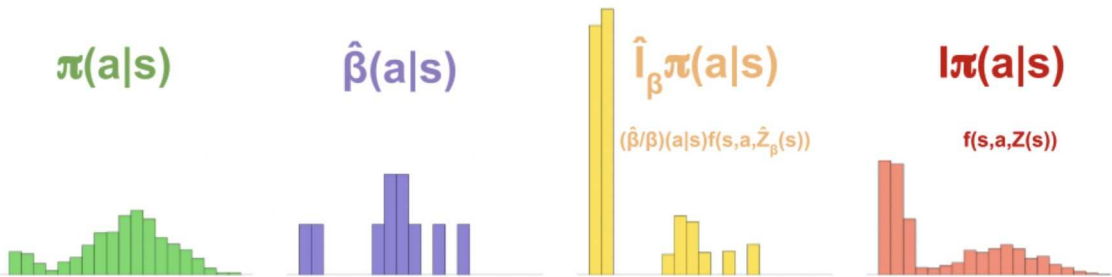
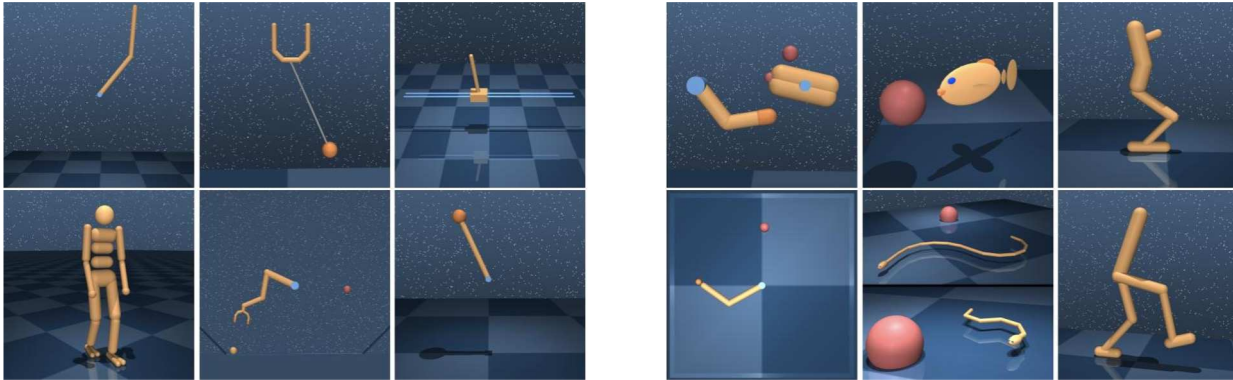


Figure 1. **Sample-based Policy Improvement.** On the left, the current policy $\pi(a|s)$. Next, K actions $\{a_i\}$ are sampled from a proposal distribution β and $\hat{\beta}(a|s)$ is the corresponding empirical distribution. A sample-based improved policy $\hat{\mathcal{I}}_{\beta}\pi(a|s) = (\hat{\beta}/\beta)(a|s)f(s,a,\hat{Z}_{\beta}(s))$ is then built. As the number of samples K increases $\hat{\mathcal{I}}_{\beta}\pi(a|s)$ converges to the improved policy $\mathcal{I}\pi(a|s) = f(s,a,Z(s))$.

Tasks	Dreamer	SMuZero
acrobot.swingup	365.26	417.52
cartpole.balance	979.56	984.86
cartpole.balance_sparse	941.84	998.14
cartpole.swingup	833.66	868.87
cartpole.swingup_sparse	812.22	846.91
cheetah.run	894.56	914.39
ball_in_cup.catch	962.48	977.38
finger.spin	498.88	986.38
finger.turn_easy	825.86	972.53
finger.turn_hard	891.38	963.07
hopper.hop	368.97	528.24
hopper.stand	923.72	926.50
pendulum.swingup	833.00	837.76
quadruped.run	888.39	923.54
quadruped.walk	931.61	933.77
reacher.easy	935.08	982.26
reacher.hard	817.05	971.53
walker.run	824.67	931.06
walker.stand	977.99	987.79
walker.walk	961.67	975.46

Applications: Offline RL

Algorithm 1 The *Reanalyse* algorithm.

```
for step  $\leftarrow 0 \dots N$  do
   $t \sim \text{random}(1 : T)$ 
   $s_t = \text{representation}(o_{1:t}, \theta)$ 
  for  $i \leftarrow 0 \dots k$  do
     $\pi_t^i, \nu_t^i = \text{improve}(\text{representation}(o_{1:t+i}, \theta), \theta)$ 
     $p_t^i, v_t^i = \text{predict}(s_t^i, \theta)$ 
     $r_t^{i+1}, s_t^{i+1} = \text{dynamics}(s_t^i, \theta)$ 
  end for
   $l = \text{loss}(h_{t:t+k}, \{r, p, v, u, \pi, \nu\}_t^{0:k}, \theta)$ 
   $\Delta\theta = \text{optimise}(l, \theta)$ 
end for
```

Game	QR-DQN	REM	CQL(H)	MZ
asterix (1%)	166.3	386.5	592.4	27220.5
breakout	7.9	11.0	61.1	251.9
pong	-13.8	-6.9	19.3	-16.2
qbert	383.6	343.4	14012.0	6953.2
sequest	672.9	499.8	779.4	4964.0
asterix (10%)	189.2	75.1	156.3	40554.0
breakout	151.2	86.7	269.3	485.8
pong	15.1	8.9	18.5	15.6
qbert	7091.3	8624.3	13855.6	16817.9
sequest	2984.8	3936.6	3674.1	8556.3

Table 3. **Low-data Atari setting.** QR-DQN (Dabney et al., 2018), REM (Agarwal et al., 2020), CQL(H) (Kumar et al., 2020) and MuZero Unplugged results when trained on only 1% (top, 2 million frames) or 10% (bottom, 20 million frames) of Atari data. MuZero Unplugged performance improves consistently when trained on more data.

Online and Offline Reinforcement Learning by Planning with a Learned Model

Hypertree proof search for neural theorem proving

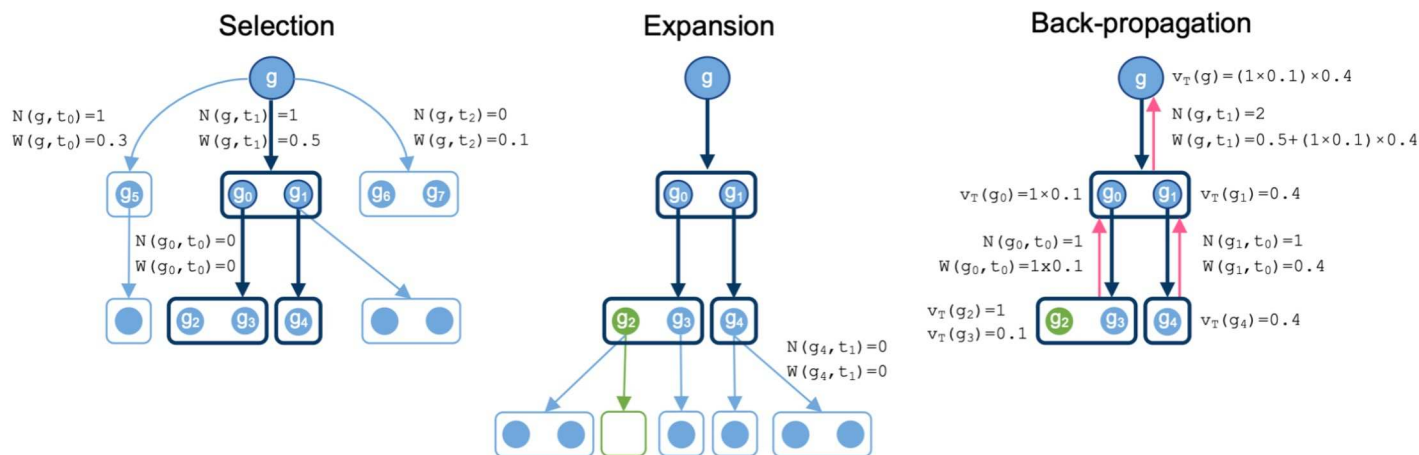
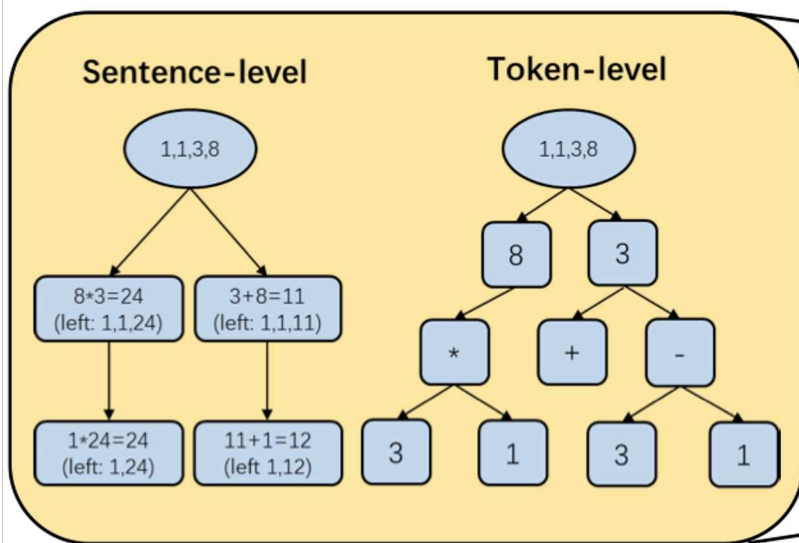


Figure 5: HyperTree Proof Search. We aim at finding a proof of the root theorem g with HTPS. Proving either $\{g_5\}$, $\{g_0, g_1\}$, or $\{g_6, g_7\}$ would lead to a proof of g by tactic t_0, t_1 , or t_2 . The figure represents the three steps of HTPS that are repeated until a proof is found. Guided by the search policy, we select a hypertree whose leaves are unexpanded nodes. The selected nodes are then expanded, adding new tactics and nodes to the hypergraph. Finally, during back-propagation we evaluate the node values of the hypertree, starting from the leaves back to the root, and update the visit counts and total action values.

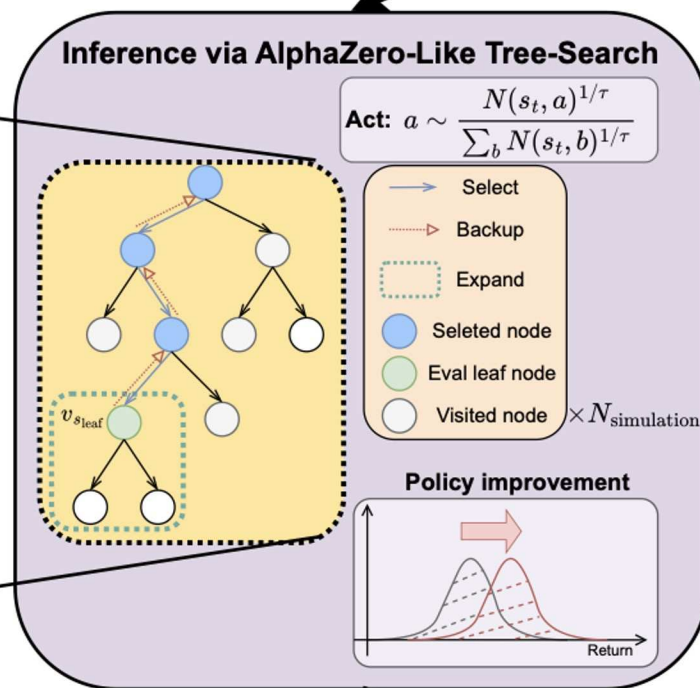
Online training statements	Supervised	GPT-f	Evariste miniF2F-valid
miniF2F-valid	38.5	47.3	58.6 [†]
miniF2F-test	35.3	36.6	41.0
miniF2F-curriculum	20.8	30.6	32.1
Train time (A100 days)	50	2000	1360

AlphaZero для LLM

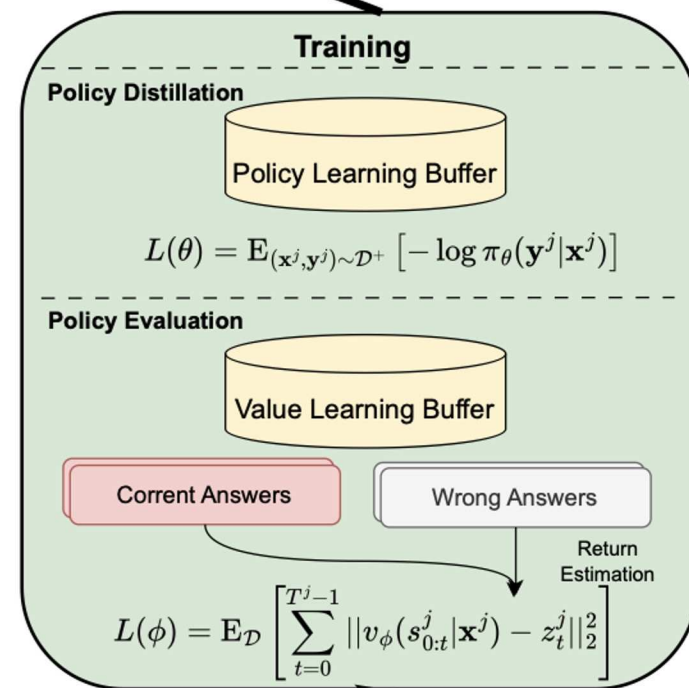
Node Expansion



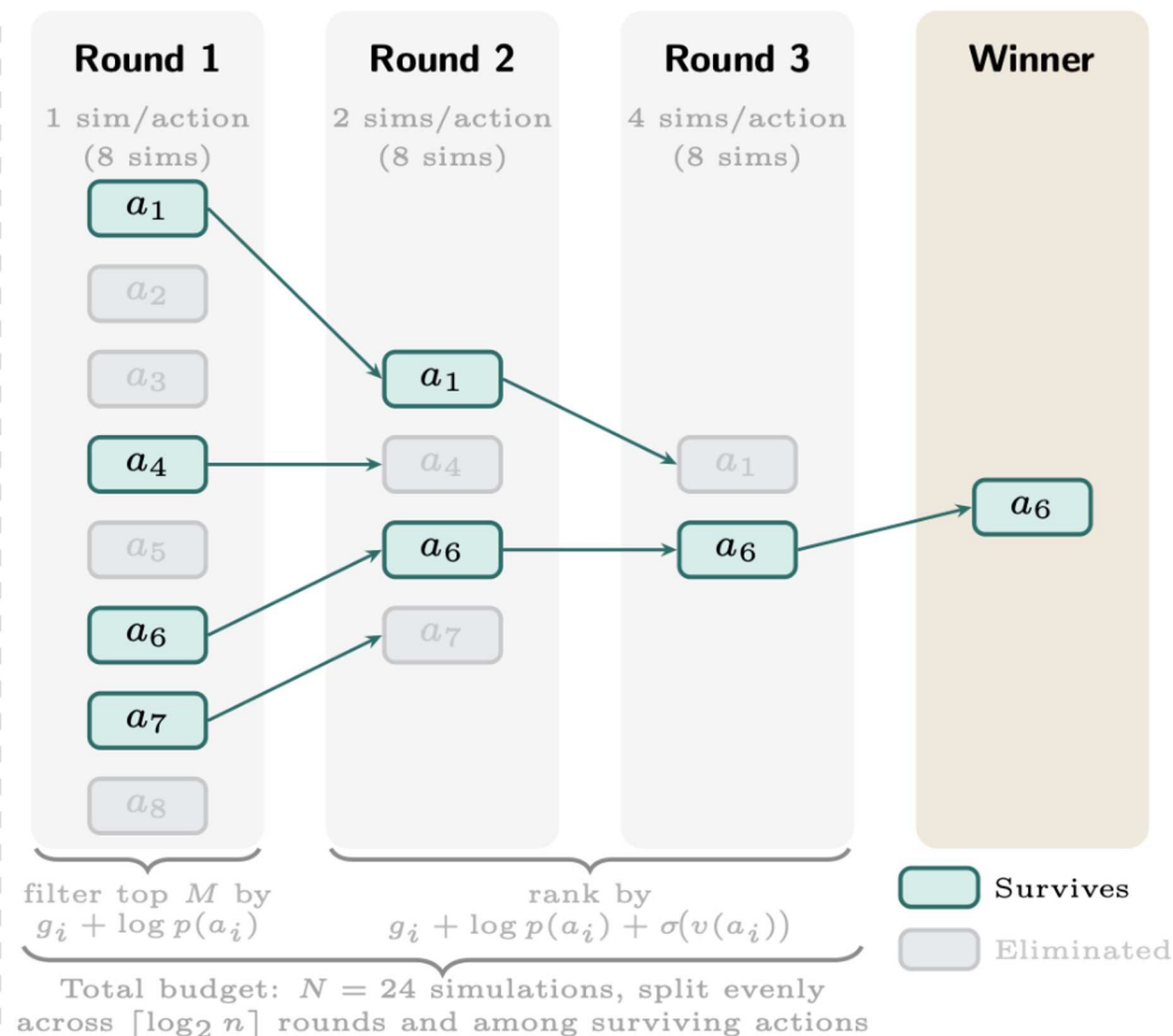
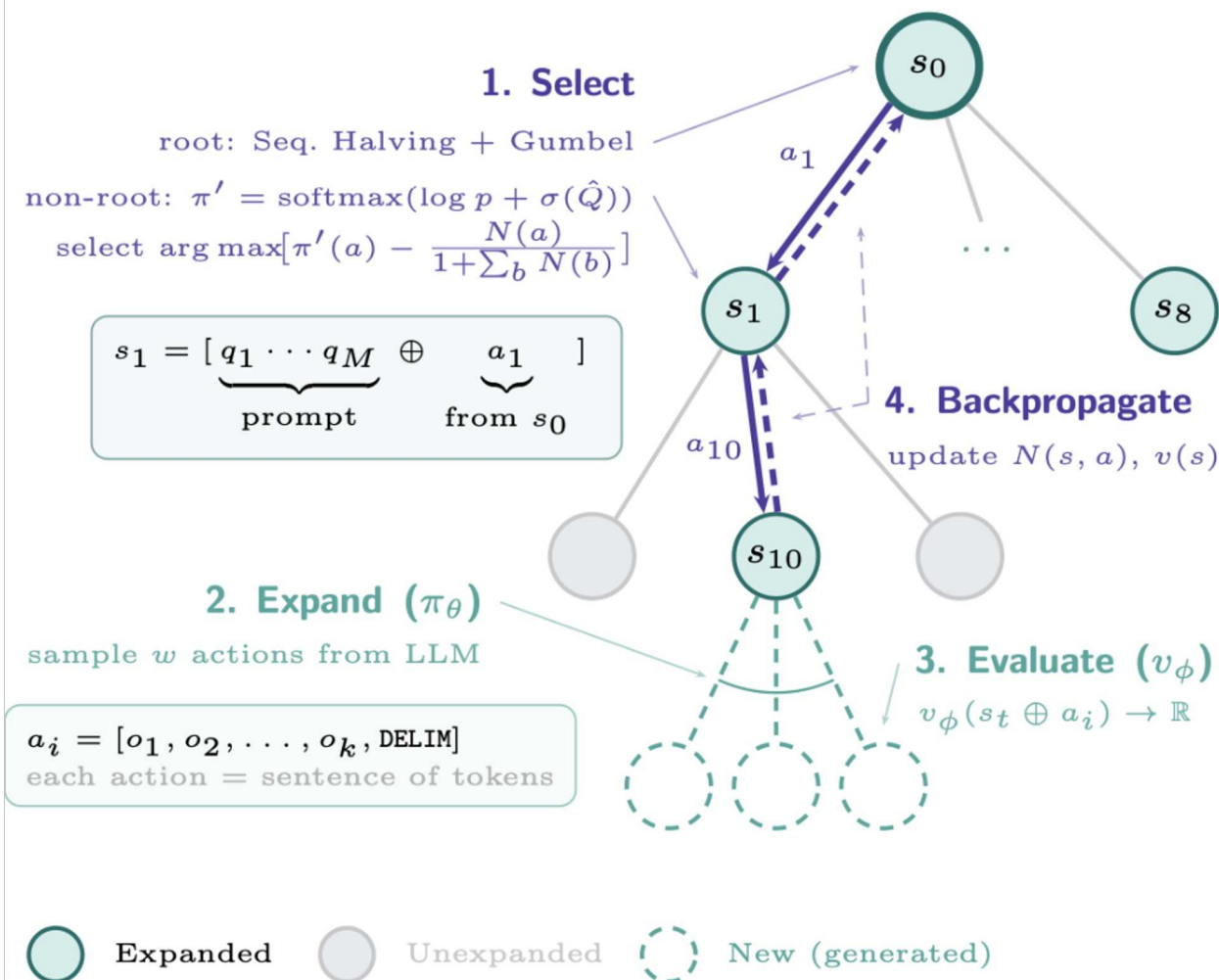
TS-LLM Enhances LLM Inference Decoding by Tree-Search



TS-LLM Provides Improved Paths for LLM Training

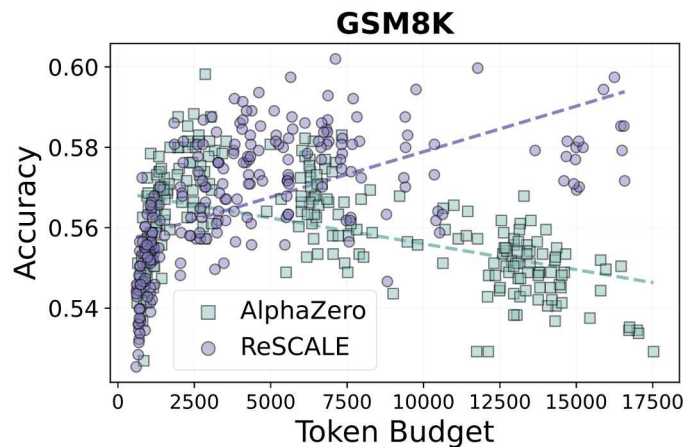


Revisiting AlphaZero для LLM



Revisiting AlphaZero для LLM

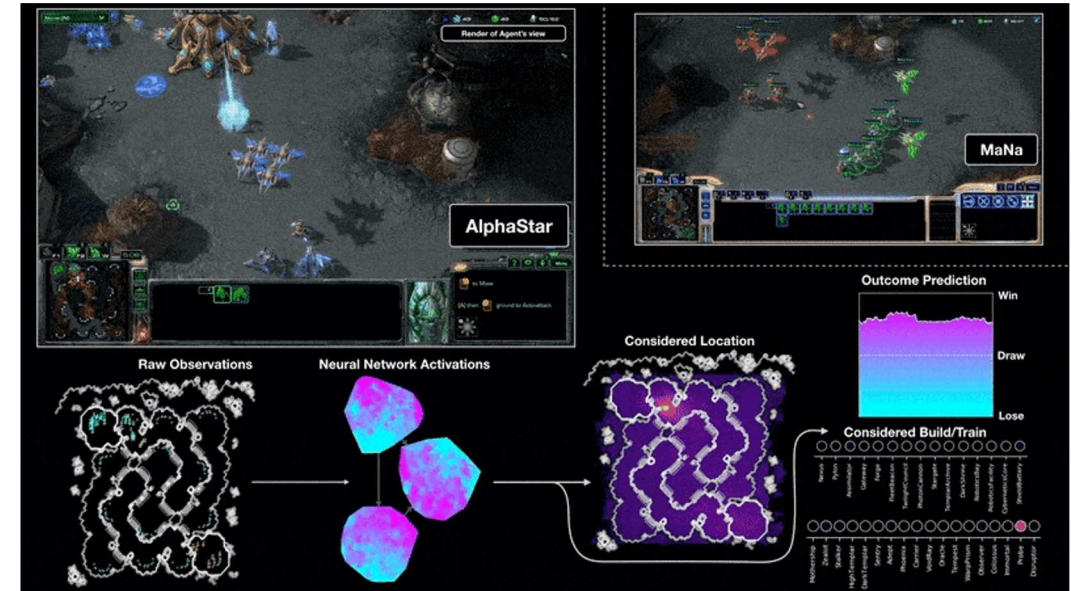
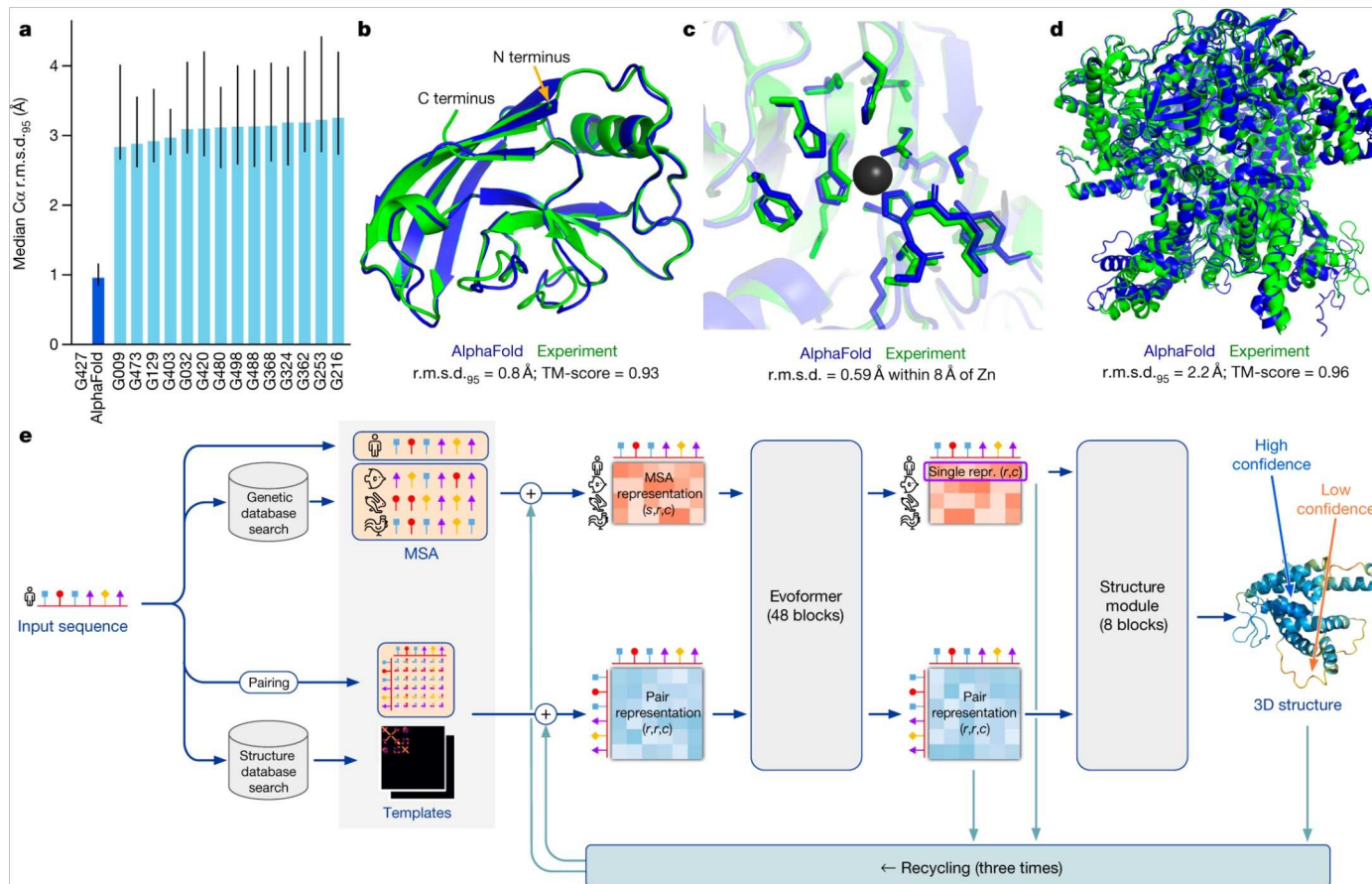
Takeaway: We analyzed MCTS trees for examples where AlphaZero MCTS produced incorrect answers and found that it overexploits a small subset of actions, leading to a large imbalance in visit counts between explored and under-explored nodes. This issue worsens with larger budgets, as root selection favors the most visited node and ignores potentially better alternatives. ReSCALE mitigates this by using Sequential Halving to preserve exploration diversity at each step.



Tree Search	Budget	GSM8K			Game24		
		Tokens	Acc., %	Max. Acc., %	Tokens	Acc., %	Max. Acc., %
AlphaZero	Small	0.5K–2K	56.0 ± 1.2	58.8	0.2K–2K	74.4 ± 12.0	86.7
ReSCALE			55.1 ± 1.1	58.6		71.6 ± 10.5	81.2
AlphaZero	Medium	6K–8K	56.7 ± 0.9	58.3	2K–4K	84.3 ± 1.0	86.2
ReSCALE			57.9 ± 1.0	60.2		83.4 ± 1.4	85.9
AlphaZero	Large	16K–18K	53.6 ± 0.7	55.0	4K–6K	82.9 ± 1.1	84.8
ReSCALE			58.4 ± 0.9	59.7		85.3 ± 0.6	85.9
Best-of-N	N = 32	3.5K	53.4 ± 0.4	-	2.5K	54.1 ± 1.0	-

Ложные AlphaZero

AlphaFold и AlphaStar – не AlphaZero :)



Open Source Implementations: EfficientZero

Game	Full	w.o. consistency	w.o. value prefix	w.o. off-policy correction
Normed Mean	1.943	0.881	1.482	1.475
Normed Median	1.090	0.340	0.552	0.836

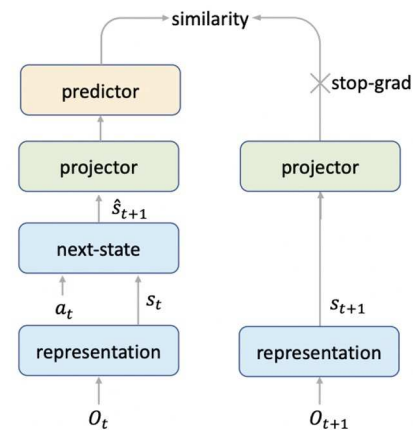
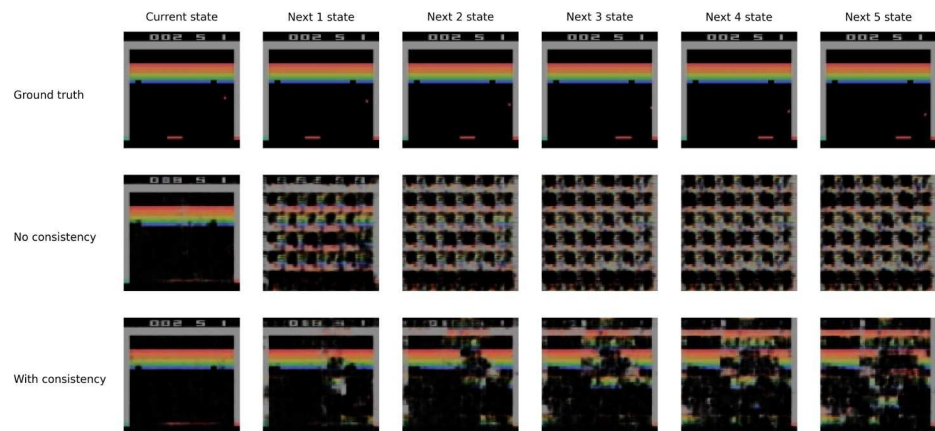
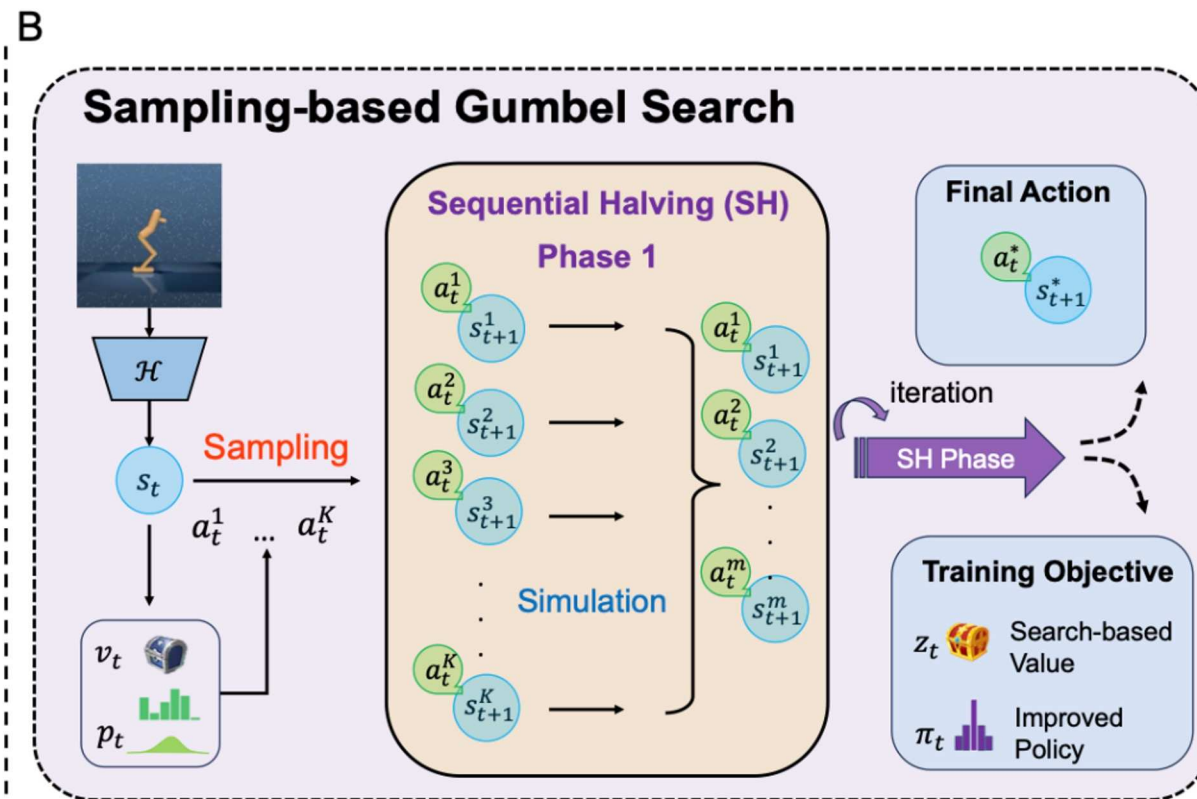
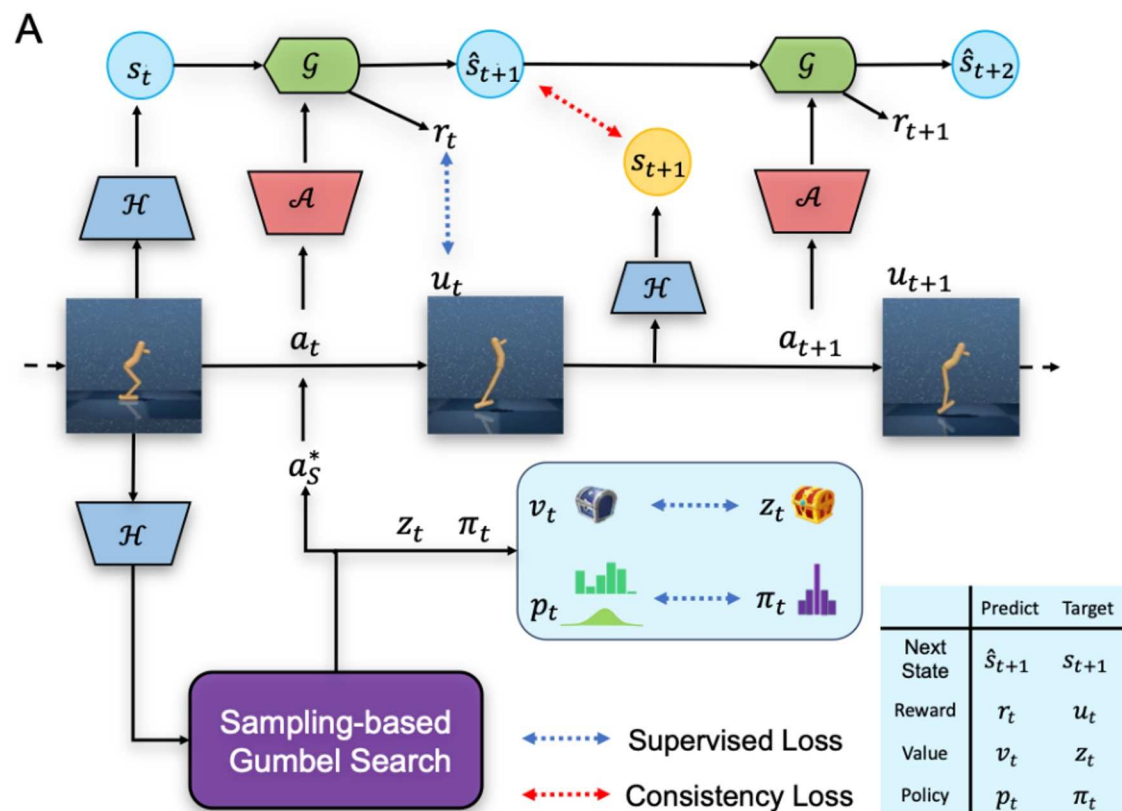


Figure 2: The self-supervised consistency loss.

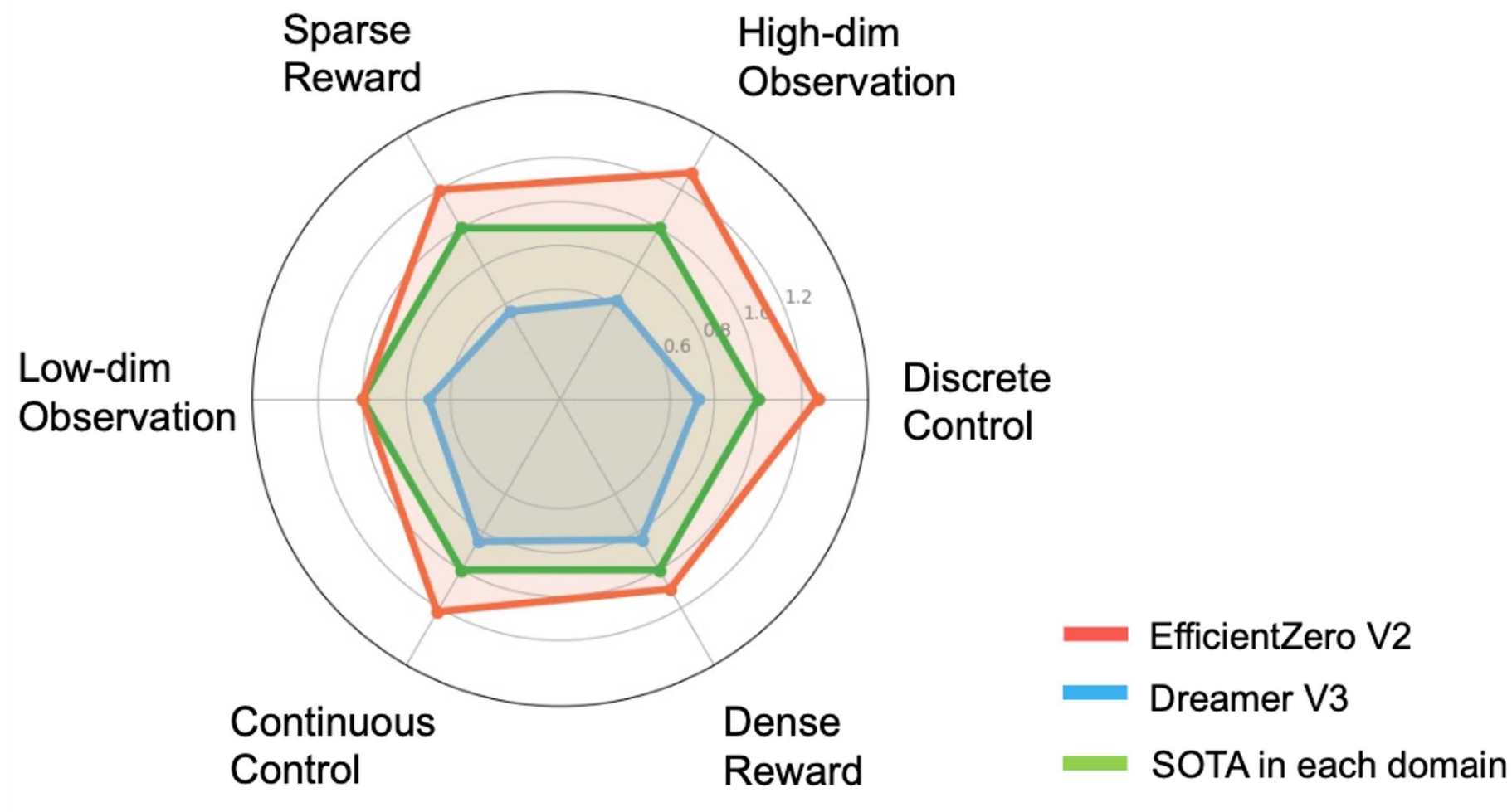
Game	Random	Human	SimPLe	OTRainbow	CURL	DrQ	SPR	MuZero	Ours
Alien	227.8	7127.7	616.9	824.7	558.2	771.2	801.5	530.0	808.5
Amidar	5.8	1719.5	88.0	82.8	142.1	102.8	176.3	38.8	148.6
Assault	222.4	742.0	527.2	351.9	600.6	452.4	571.0	500.1	1263.1
Asterix	210.0	8503.3	1128.3	628.5	734.5	603.5	977.8	1734.0	25557.8
Bank Heist	14.2	753.1	34.2	182.1	131.6	168.9	380.9	192.5	351.0
BattleZone	2360.0	37187.5	5184.4	4060.6	14870.0	12954.0	16651.0	7687.5	13871.2
Boxing	0.1	12.1	9.1	2.5	1.2	6.0	35.8	15.1	52.7
Breakout	1.7	30.5	16.4	9.8	4.9	16.1	17.1	48.0	414.1
ChopperCmd	811.0	7387.8	1246.9	1033.3	1058.5	780.3	974.8	1350.0	1117.3
Crazy Climber	10780.5	35829.4	62583.6	21327.8	12146.5	20516.5	42923.6	56937.0	83940.2
Demon Attack	152.1	1971.0	208.1	711.8	817.6	1113.4	545.2	3527.0	13003.9
Freeway	0.0	29.6	20.3	25.0	26.7	9.8	24.4	21.8	21.8
Frostbite	65.2	4334.7	254.7	231.6	1181.3	331.1	1821.5	255.0	296.3
Gopher	257.6	2412.5	771.0	778.0	669.3	636.3	715.2	1256.0	3260.3
Hero	1027.0	30826.4	2656.6	6458.8	6279.3	3736.3	7019.2	3095.0	9315.9
Jamesbond	29.0	302.8	125.3	112.3	471.0	236.0	365.4	87.5	517.0
Kangaroo	52.0	3035.0	323.1	605.4	872.5	940.6	3276.4	62.5	724.1
Krull	1598.0	2665.5	4539.9	3277.9	4229.6	4018.1	3688.9	4890.8	5663.3
Kung Fu Master	258.5	22736.3	17257.2	5722.2	14307.8	9111.0	13192.7	18813.0	30944.8
Ms Pacman	307.3	6951.6	1480.0	941.9	1465.5	960.5	1313.2	1265.6	1281.2
Pong	-20.7	14.6	12.8	1.3	-16.5	-8.5	-5.9	-6.7	20.1
Private Eye	24.9	69571.3	58.3	100.0	218.4	-13.6	124.0	56.3	96.7
Qbert	163.9	13455.0	1288.8	509.3	1042.4	854.4	669.1	3952.0	13781.9
Road Runner	11.5	7845.0	5640.6	2696.7	5661.0	8895.1	14220.5	2500.0	17751.3
Seaquest	68.4	42054.7	683.3	286.9	384.5	301.2	583.1	208.0	1100.2
Up N Down	533.4	11693.2	3350.3	2847.6	2955.2	3180.8	28138.5	2896.9	17264.2
Normed Mean	0.000	1.000	0.443	0.264	0.381	0.357	0.704	0.562	1.943
Normed Median	0.000	1.000	0.144	0.204	0.175	0.268	0.415	0.227	1.090

Mastering Atari Games with Limited Data

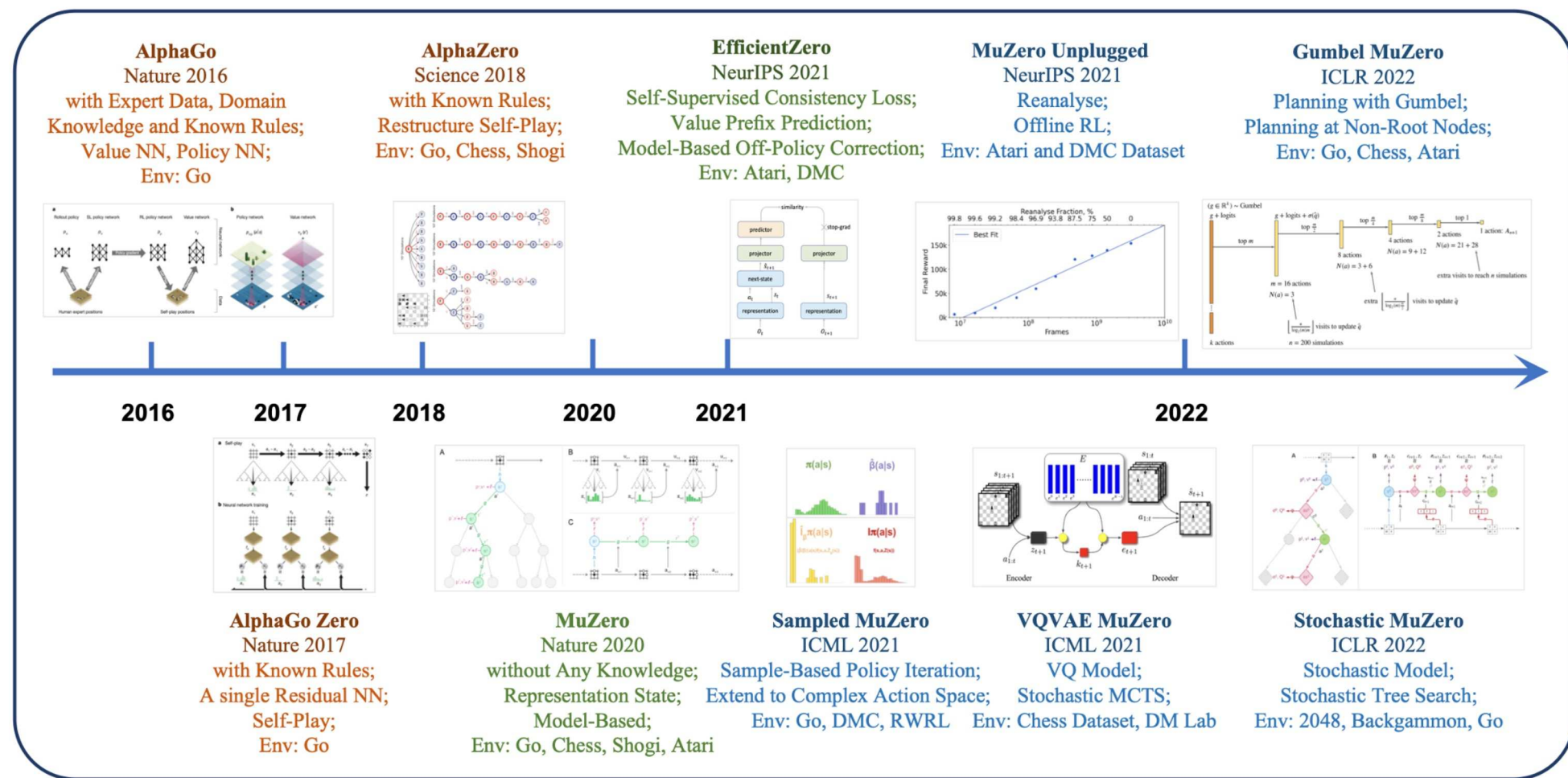
Open Source Implementations: EfficientZero-v2



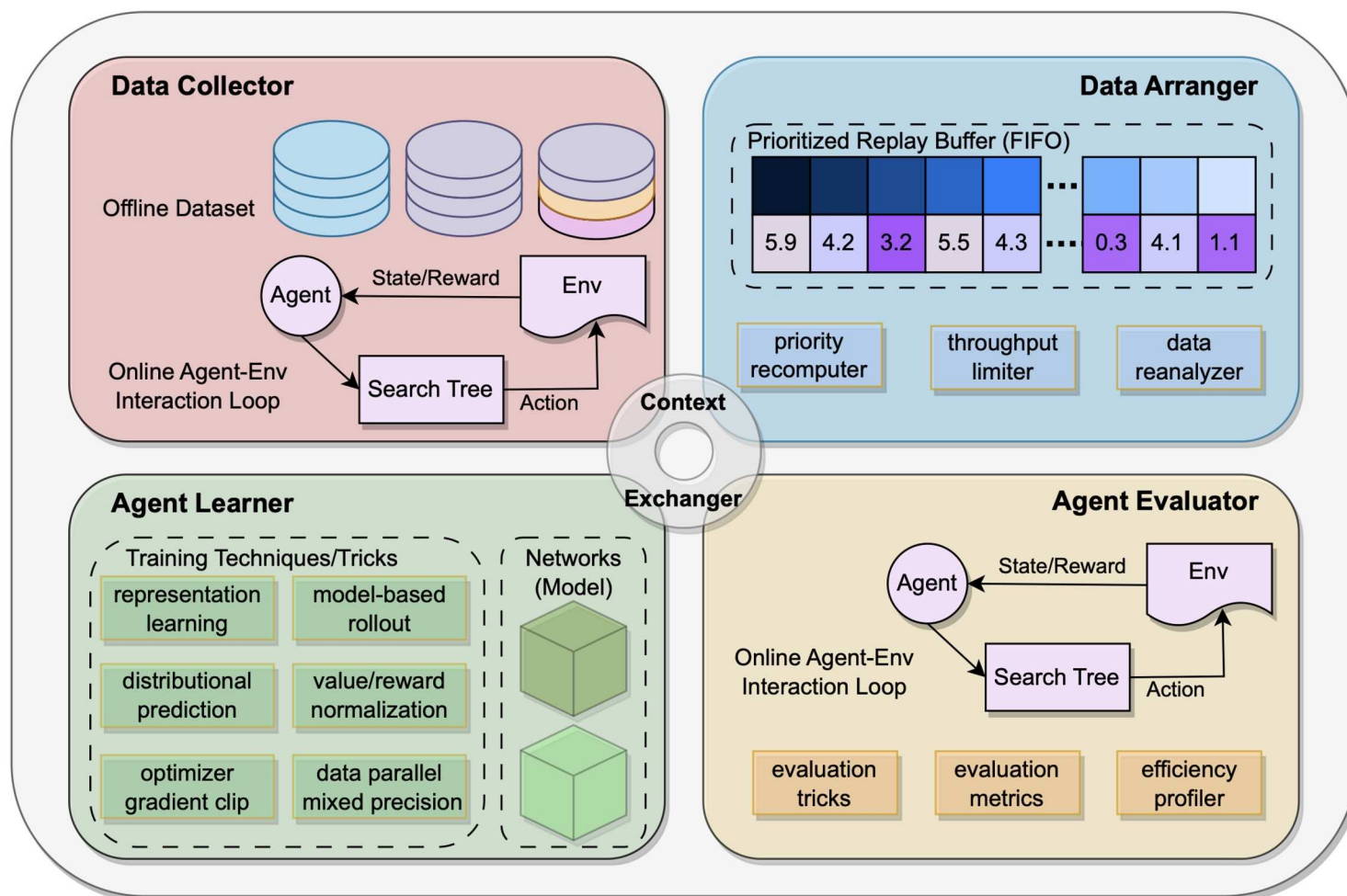
Open Source Implementations: EfficientZero-v2



LightZero



LightZero



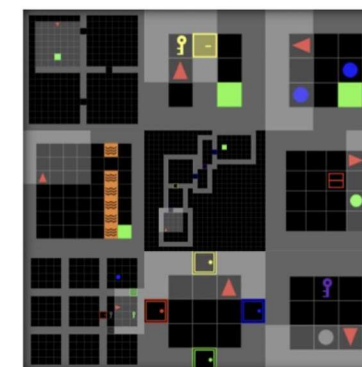
(a) Board Games



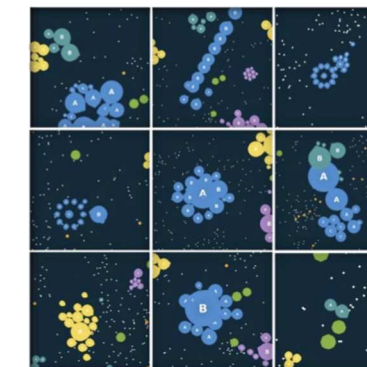
(b) Atari



(c) MiniGrid



(d) GoBigger



Open Source Implementations: Lc0

Top Chess Engine Championship (TCEC)^[43]

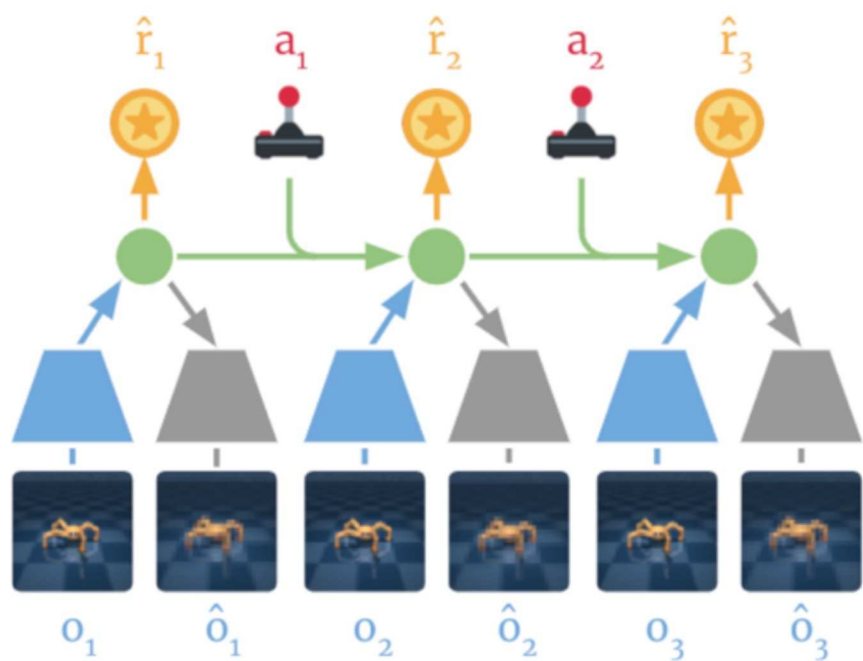
Season	Division 4	Division 3	Division 2	Division 1	Division P	Infrafinal	Superfinal
12 (2018)	8th	—	—	—	—	—	—
13 (2018)	1st	3rd	—	—	—	—	—
14 (2018)	—	1st	1st	1st	2nd	—	2nd
15 (2019)	—	—	—	—	2nd	—	1st
16 (2019)	—	—	—	—	3rd	—	—
17 (2020)	—	—	—	—	1st	—	1st
18 (2020)	—	—	—	—	2nd	—	2nd
19 (2020)	—	—	—	—	2nd	—	2nd
20 (2020)	—	—	—	—	1st	—	2nd
21 (2021)	—	—	—	—	2nd	—	2nd
22 (2022)	—	—	—	—	3rd	3rd	—



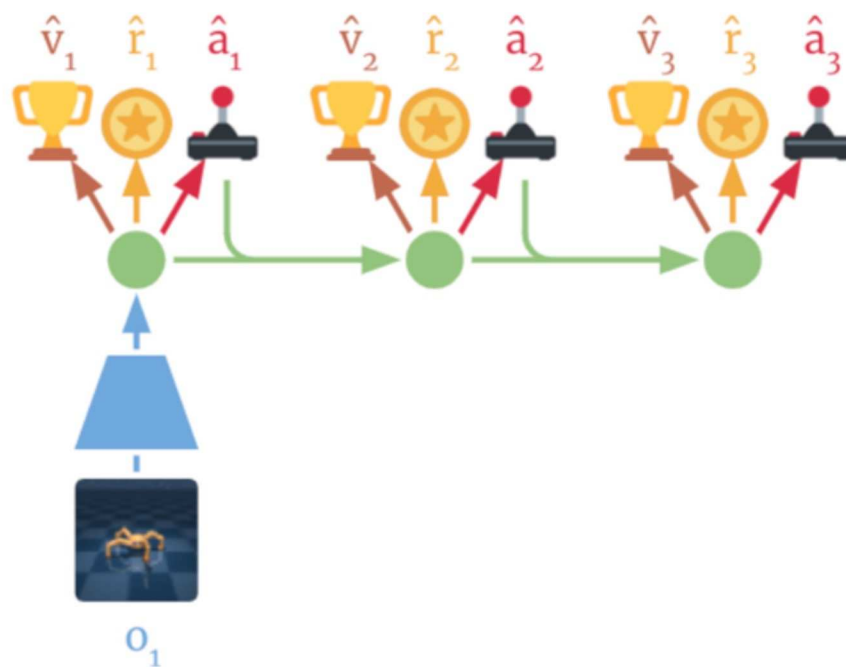
05

Dreamer

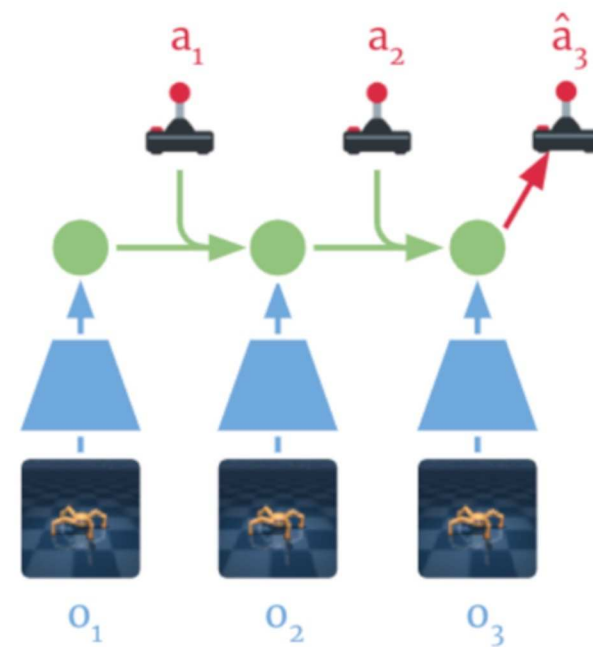
Dreamer



Learn dynamics from dataset



Learn behavior in imagination



Environment interaction

Псевдокод Dreamer-v1

Algorithm 1: Dreamer

Initialize dataset $\mathcal{D}$ with S random seed episodes.
Initialize neural network parameters θ, ϕ, ψ randomly.

while not converged do

for update step $c = 1..C$ **do**

// Учим динамику среды

Draw B data sequences $\{(a_t, o_t, r_t)\}_{t=k}^{k+L} \sim \mathcal{D}$.

Compute model states $s_t \sim p_\theta(s_t | s_{t-1}, a_{t-1}, o_t)$.

Update θ using representation learning.

// Учим поведение агента

Imagine trajectories $\{(s_\tau, a_\tau)\}_{\tau=t}^{t+H}$ from each s_t .

Predict rewards $E(q_\theta(r_\tau | s_\tau))$ and values $v_\psi(s_\tau)$.

Compute value estimates $V_\lambda(s_\tau)$ via Equation 6.

Update $\phi \leftarrow \phi + \alpha \nabla_\phi \sum_{\tau=t}^{t+H} V_\lambda(s_\tau)$.

Update $\psi \leftarrow \psi - \alpha \nabla_\psi \sum_{\tau=t}^{t+H} \frac{1}{2} \|v_\psi(s_\tau) - V_\lambda(s_\tau)\|^2$.

// Взаимодействие со средой и сбор данных

$o_1 \leftarrow \text{env.reset}()$

for time step $t = 1..T$ **do**

Compute $s_t \sim p_\theta(s_t | s_{t-1}, a_{t-1}, o_t)$ from history. Восстанавливаем состояние по истории

Compute $a_t \sim q_\phi(a_t | s_t)$ with the action model. Выбираем действие

Add exploration noise to action.

$r_t, o_{t+1} \leftarrow \text{env.step}(a_t)$.

Add experience to dataset $\mathcal{D} \leftarrow \mathcal{D} \cup \{(o_t, a_t, r_t)_{t=1}^T\}$.

Model components

Представление $p_\theta(s_t | s_{t-1}, a_{t-1}, o_t)$

Функция перехода $q_\theta(s_t | s_{t-1}, a_{t-1})$

Функция вознаграждения $q_\theta(r_t | s_t)$

Действия $q_\phi(a_t | s_t)$

Полезность $v_\psi(s_t)$

Hyper parameters

Seed episodes S

Collect interval C

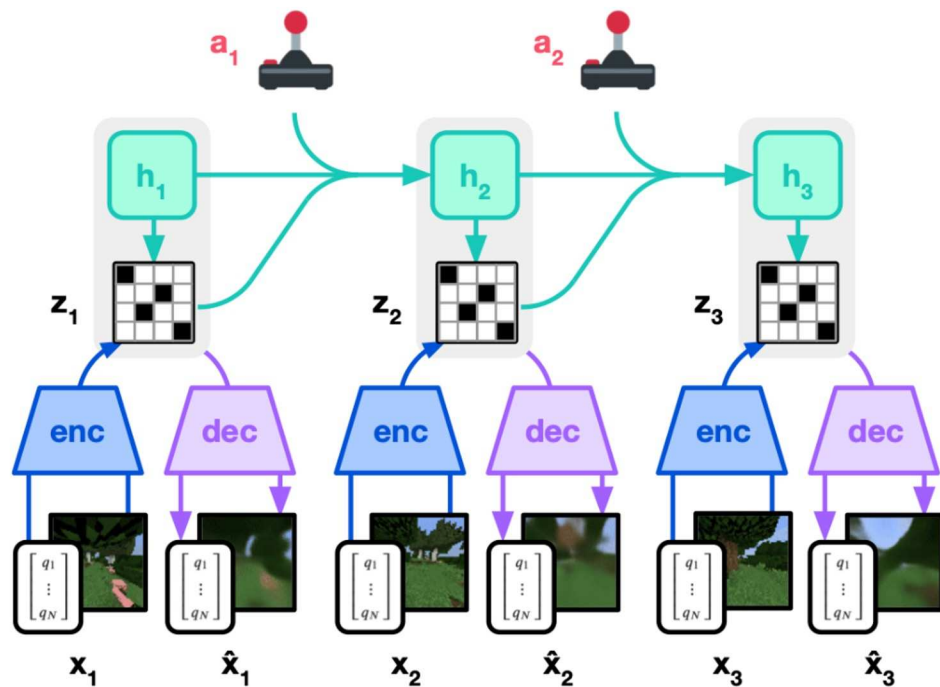
Batch size B

Sequence length L

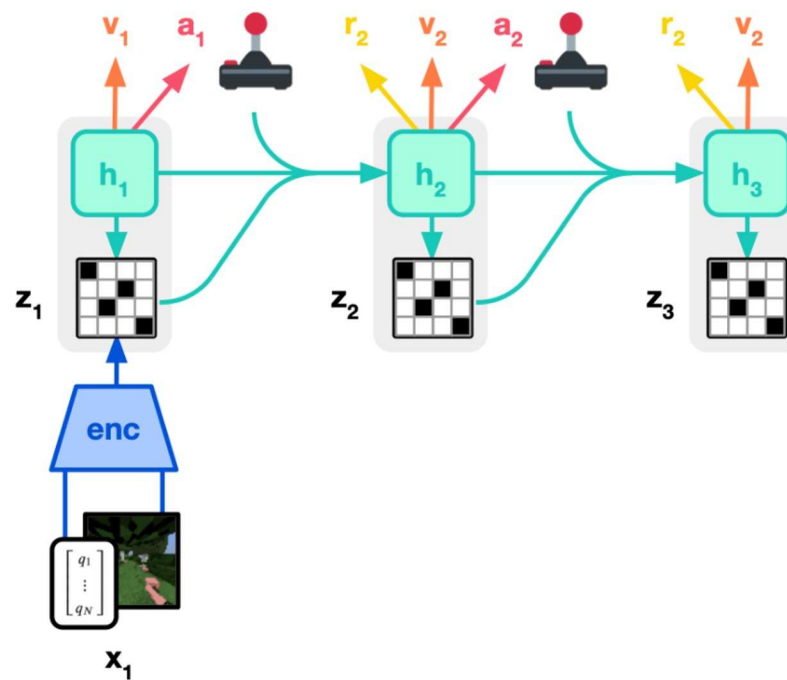
Imagination horizon H

Learning rate α

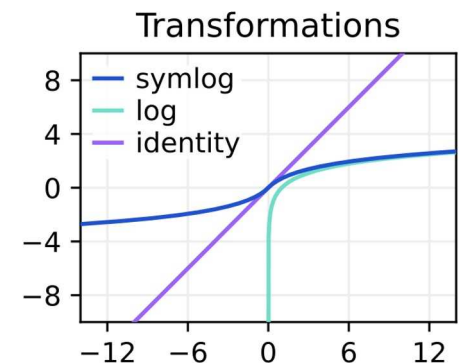
Dreamer-v3



(a) World Model Learning



(b) Actor Critic Learning

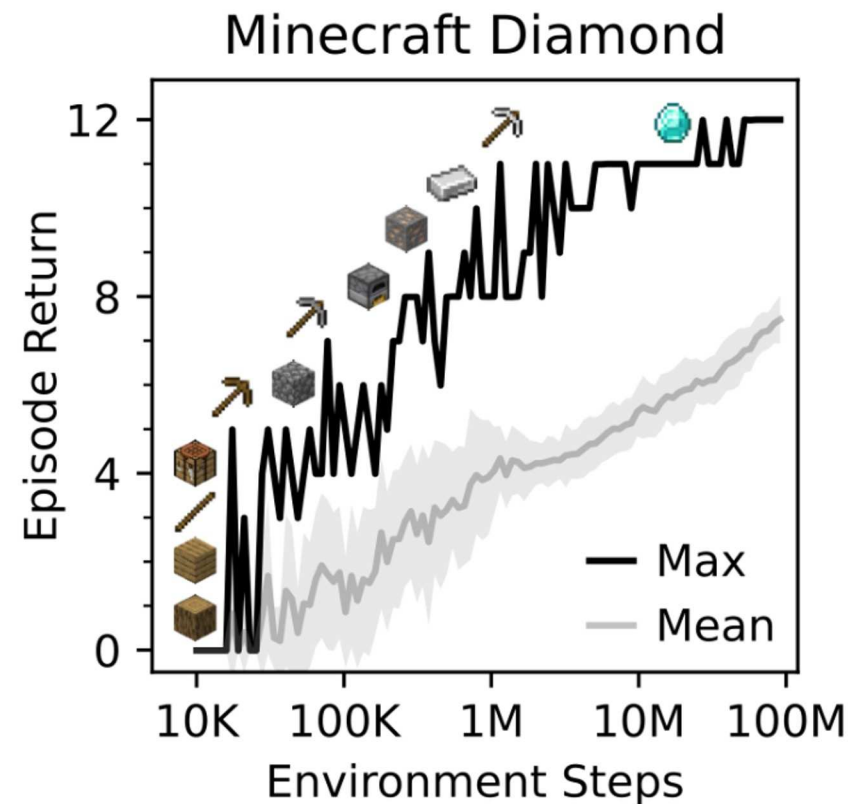
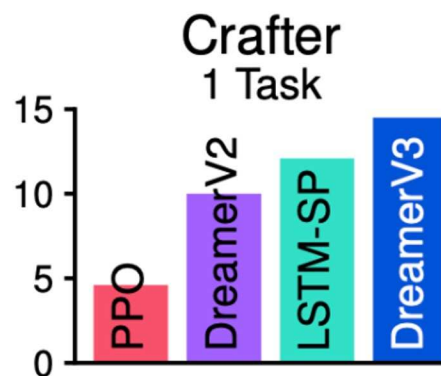
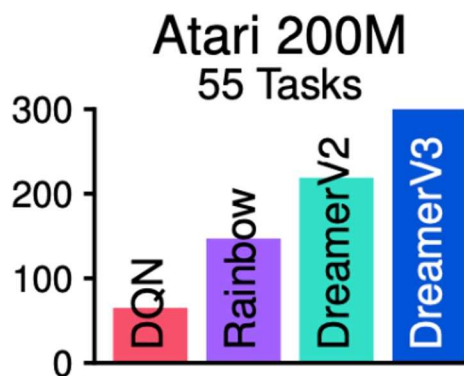
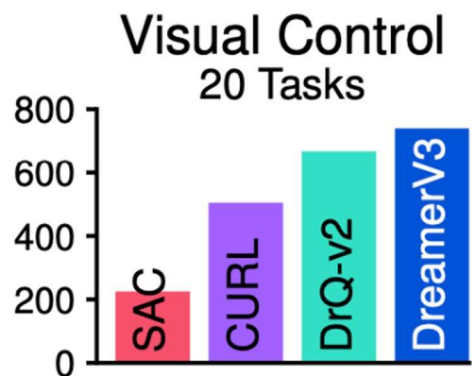
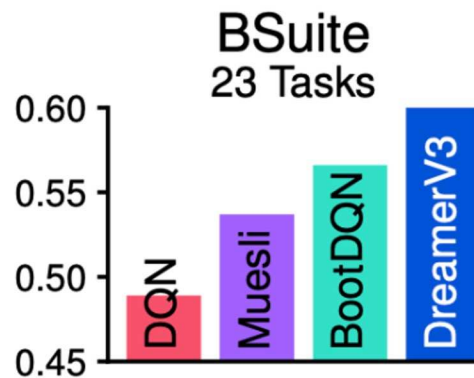
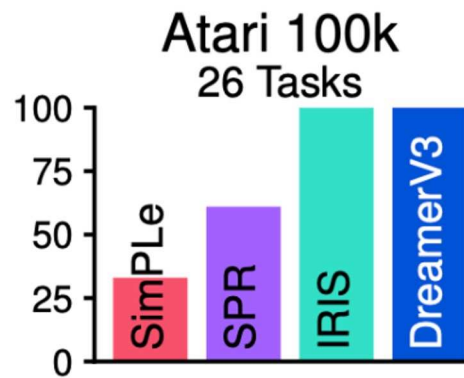
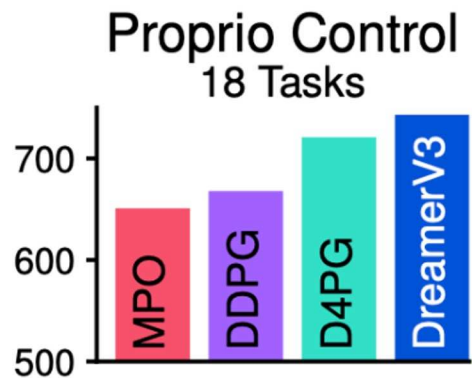


Symlog используется для того, чтобы решить проблему разного масштаба вознаграждений, от высоких и разреженных до низких и частых. — Способ нормализации.

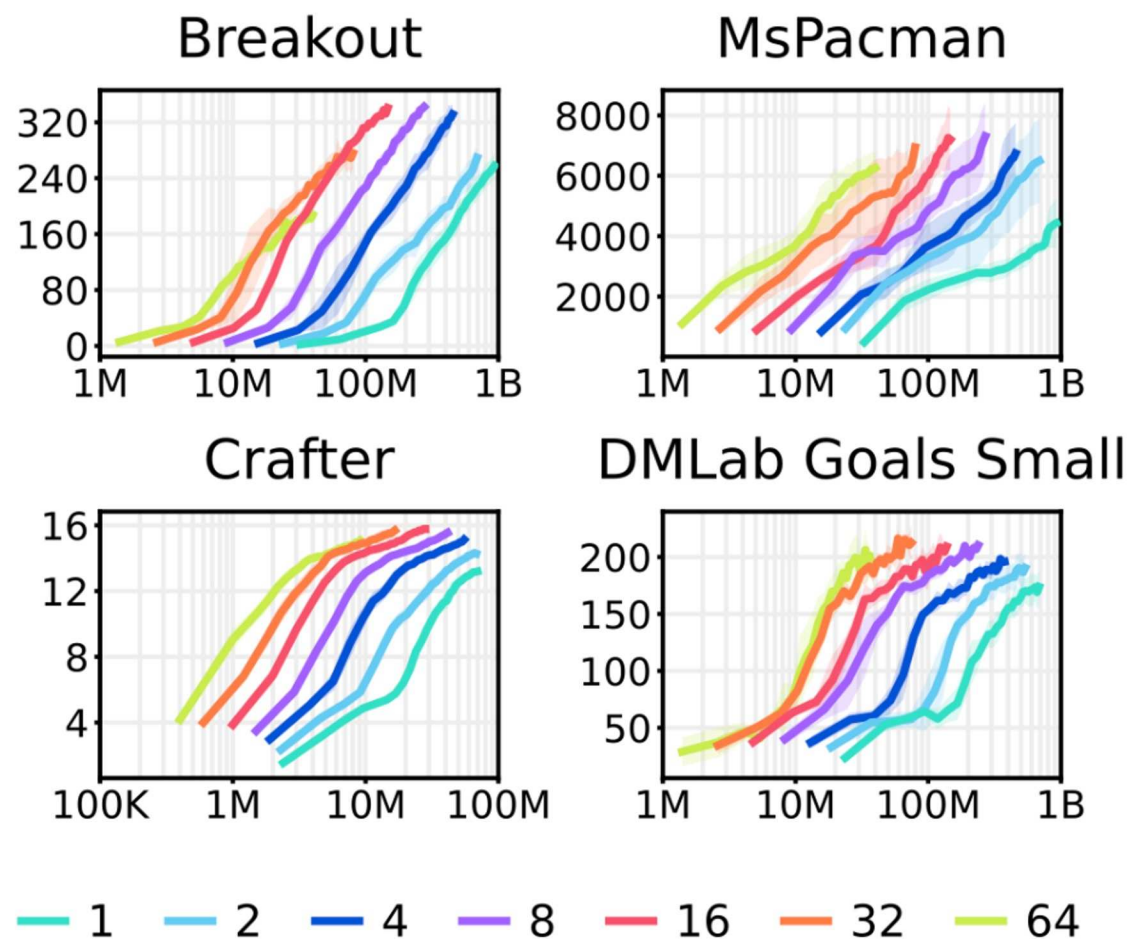
$$\text{symlog}(x) \doteq \text{sign}(x) \ln(|x| + 1)$$

Figure 3: Training process of DreamerV3. The world model encodes sensory inputs into a discrete representation z_t that is predicted by a sequence model with recurrent state h_t given actions a_t . The inputs are reconstructed as learning signal to shape the representations. The actor and critic learn from trajectories of abstract representations predicted by the world model.

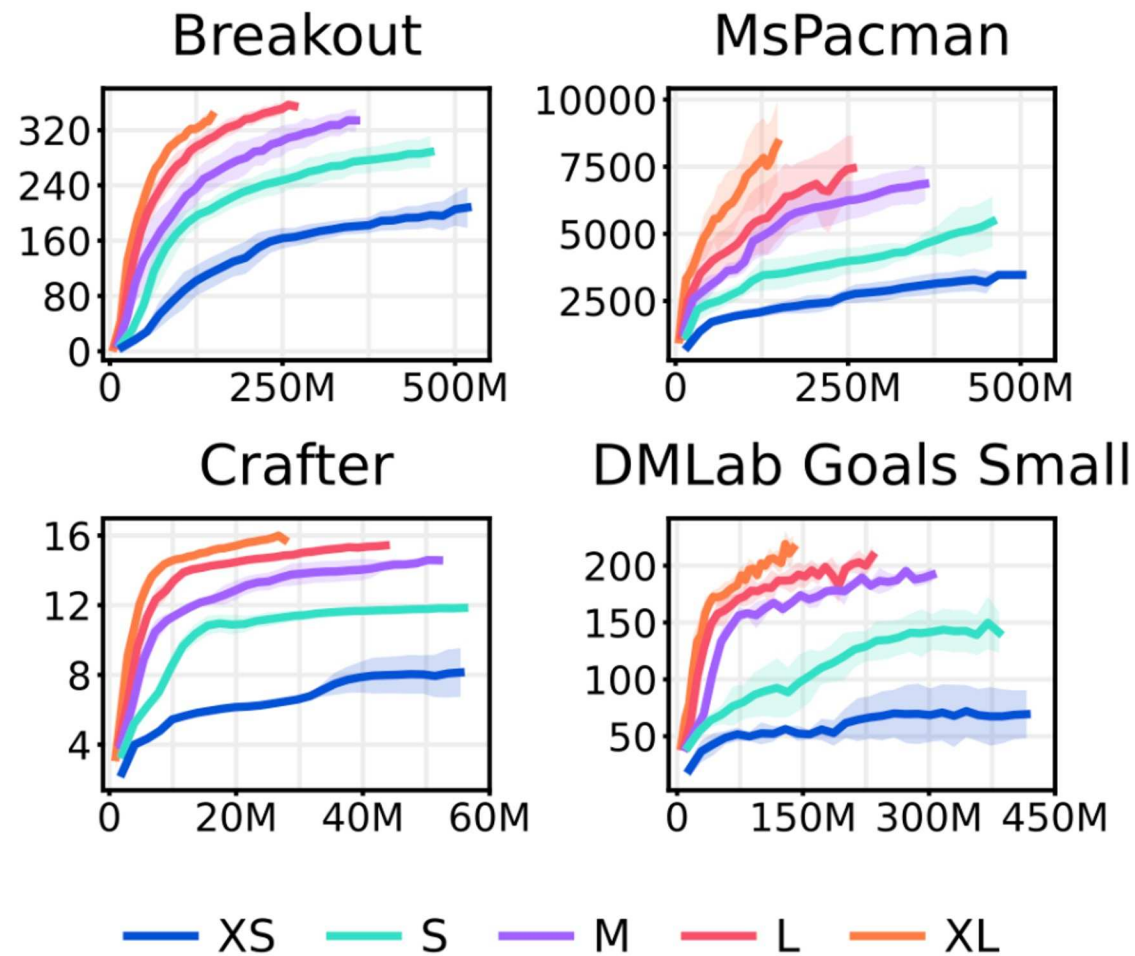
Результаты



Увеличение модели



(a) Training Ratio



(b) Model Size

Резюме:

Crafter



APPO agent was trained for ~500M environment steps in under 24 hours on a single GPU:

- Сэмпл-эффективный алгоритм
- Низкая вычислительная эффективность (если смотрим не на шаги в среде, а на GPU часы)
- Dreamer-V3 можно применять почти для любой среды, без каких-либо доп. изменений
- Хороший выбор для медленных сред

Achievement	Rainbow	PPO	DreamerV2	APPO
Collect Coal	0.0%	0.4%	14.7%	96.6%
Collect Diamond	0.0%	0.0%	0.0%	22.6%
Collect Drink	24.0%	30.3%	80.0%	94.0%
Collect Iron	0.0%	0.0%	0.0%	83.1%
Collect Sapling	97.4%	66.7%	86.6%	98.9%
Collect Stone	0.2%	3.0%	42.7%	99.3%
Collect Wood	74.9%	83.0%	92.7%	99.9%
Defeat Skeleton	0.7%	0.2%	2.6%	89.3%
Defeat Zombie	39.6%	2.0%	53.1%	95.2%
Eat Cow	26.1%	12.0%	17.1%	93.3%
Eat Plant	0.0%	0.0%	0.1%	1.0%
Make Iron Pickaxe	0.0%	0.0%	0.0%	77.9%
Make Iron Sword	0.0%	0.0%	0.0%	55.4%
Make Stone Pickaxe	0.0%	0.0%	0.2%	97.6%
Make Stone Sword	0.0%	0.0%	0.3%	98.5%
Make Wood Pickaxe	4.8%	21.1%	59.6%	99.7%
Make Wood Sword	9.8%	20.1%	40.2%	99.0%
Place Furnace	0.0%	0.1%	1.8%	98.0%
Place Plant	94.2%	65.0%	84.4%	99.0%
Place Stone	0.0%	1.7%	29.0%	99.0%
Place Table	52.3%	66.1%	85.7%	99.0%
Wake Up	93.3%	92.5%	92.8%	97.0%
Score	4.3%	4.6%	10.0%	50.0%

Вопросы?